BVCC Wallet

BLOCK VENTURE CHAIN CAPITAL

Agent Wallet Security & Test Report

Five generations, V1 to V5: review rounds, remediation, deployment and on-chain verification

SUBJECT	BVCC Agent Wallet (ERC-4337 / ERC-7579 smart account)
CODEBASE	**V5 (current):** commit `838a70f` · `contracts-v5/src` tree `64a5c674` **V4 (previous):** commit `a3e28f1` · `contracts/src` tree `337e84f0`
DEPLOYMENT	Arbitrum One · Ethereum · BNB Chain · Base · Polygon · Arbitrum Sepolia
ENGAGEMENT	**Internal** security review, V5 rounds run with AI analysis tools. Not an external audit
PERIOD	2026-06-06 → 2026-11-11 · V1 to V4, then five V5 rounds and a red-team exercise
FINDINGS	2 Critical · 13 High · 10 Medium · 3 Low · 2 Informational
STATUS (V5)	26 closed · 4 accepted · 0 open · 191/191 tests · 72/72 contracts verified

Contents

PART I — AUDIT

1	Executive Summary	Headline results and what a reader should take away
2	Scope, Methodology & Severity Model	What was reviewed, at which commit, how severity is assigned
3	Findings Register	Every finding, its severity and its status, on one page
4	Findings in Detail	BVCC-01 … BVCC-13 (V4 review) · BVCC-14 … BVCC-30 (V5 review)
5	Remediation & Deployment Verification	V5 and V4: what is live, on which networks, verified how
6	Residual Risk & Recommendations	What is still exposed and what to do about it
7	Conclusion & Limitations	Assessment, and what this review does not establish

PART II — APPENDICES

A	Security Architecture	V5 design first; V3/V4 call policies and validator registry
B	Test Evidence	V5 suite and on-chain runs; V1–V4 batteries
C	Static Analysis	Slither runs and the false-positive register
D	Previously Remediated — V1 & V2	Findings closed in superseded generations, with evidence
E	Remediation Timeline	Every round from V1 to V5, compressed
F	Key Identifiers & Addresses	Contracts, networks, build
G	Process Notes	What this review got wrong, and what that implies

Findings are cited by identifier (`BVCC-01`) throughout, not by section number. Sections are cited by number, appendices by letter.

1 Executive Summary

The BVCC Agent Wallet is an ERC-4337 smart account that lets an owner delegate bounded spending authority to an autonomous agent (an EOA). This report covers every review of that delegation path, from the first permission battery in June 2026 to the V5 generation deployed on 2026-11-02. V5 is the current generation. V1 to V4 stay in the report because wallets created by their factories still exist and keep their own bytecode.

30	2	0	191
FINDINGS, V4 AND V5 REVIEWS	CRITICAL	OPEN IN V5	V5 TESTS, ALL PASSING

V5, the current generation

V5 rewrites the agent permission model instead of patching it. The V4 review left one material risk open, **BVCC-03**: an agent's token budget did not bound what it could spend through a protocol where the owner had already granted an allowance. V5 closes it by measuring what leaves the wallet. Each agent batch reads the balance of every relevant token before and after it runs and charges the difference, fee included, against the agent's limits. It does not matter how the calls are combined or which allowances already exist. Every protocol call by an agent now needs an on-chain validator. A limit of 0 means nothing is allowed and an empty recipient list means no recipient. Agent permissions expire within a year, owner signatures carry an expiry, and guardians now vote on a specific proposal, which closes the recovery stall of **BVCC-09**.

Before it was deployed, the V5 code went through five review rounds and a red-team exercise. Together they found **seventeen issues**, BVCC-14 to BVCC-30: ten High, five Medium and two Low. None of them reached a network. Thirteen were fixed in code. Four are accepted and documented as trust assumptions: validators must be immutable, approved Uniswap v4 hooks and allowed tokens are trusted, and the wallet only measures tokens a validator declares. The fifth round found nothing new and the code was frozen on 2026-10-29. Most of the High findings sit in the validators for Uniswap positions, and several of them appeared right next to the fix for the previous one; Appendix G comes back to that.

V5 has been live on Ethereum, Arbitrum One, Base, BNB Chain, Polygon and Arbitrum Sepolia since 2026-11-02. All 72 contracts (12 per network) are source-verified on the block explorers, and the 30 validators are active in the registry. Both facts were read back from each network for this edition, on 2026-11-11.

V4 and earlier: what the review found

Two findings are Critical. **BVCC-01**, a cross-function reentrancy, let an agent whose address later acquired code re-enter the owner-level `execute()` and bypass every configured limit at once; a proof of concept moved 49.925 ETH against a 0.003 ETH lifetime budget, and the same attack succeeded against a wallet created from the live V2 factory on a mainnet fork. **BVCC-02**, a wrong command mask in the Universal Router validator, would have accepted a cross-chain bridge deposit as if it were a swap — the exact exfiltration class the validator exists to prevent — and was caught in review before that validator was ever registered on a network.

Three findings are High: agent budgets do not bound protocol spending where a standing token allowance exists (**BVCC-03**, open in V4, closed in V5), guardians could be chosen by whoever first

called the permissionless factory (**BVCC-04**), and a completed recovery did not stop the agents authorized before it (**BVCC-05**). The remainder are Medium and below.

What was done about it

Nine of the V4-review findings were remediated in V4. Seven of them required a bytecode change and shipped together as the **V4** generation, deployed and independently verified on all six supported networks on 2026-07-28. Four stayed open or accepted in V4: BVCC-03, the material one, and BVCC-09, BVCC-10 and BVCC-13. V5 closes all four in code, and keeps every V4 fix.

All seventeen V5 findings were handled before deployment, with a regression test for each fix. The deep-validation layer is complete on V5 as well: the Universal Router, the Uniswap v4 PositionManager, the Aave v3 Pool, the Uniswap v3 position manager and the wrapped native token each have an active validator on every network.

> **Two caveats belong next to any statement that V5 fixed these.** First, a deployed smart account cannot be upgraded in place: its address derives from its code. **Wallets still on V1 to V4 run their own bytecode until their owners migrate.** BVCC-01 affected V1, V2 and V3 alike, and BVCC-03 is still open on every V4 wallet. The app flags an outdated wallet, and a migration guide plus a "Send everything" button move the funds in one signature per network. Second, V5's guarantees rest on four stated trust assumptions (BVCC-19, BVCC-25, BVCC-29 and BVCC-30). §6 says what each one asks of the owner.

> **This is an internal review.** The development team performed it on its own code. The five V5 rounds and the red-team exercise were run by the team with AI analysis tools; no independent human auditor has reviewed any generation, and the contracts have not been externally audited. Four of the V4 findings (BVCC-04, BVCC-06, BVCC-07 and BVCC-11) came from a later pass with a different tool after three earlier passes had missed them, and the V5 rounds kept finding new paths until the fifth. Both facts say the same thing: **absence of a finding here is weak evidence of absence.** §7 states the limits of what this document establishes.

2 Scope, Methodology & Severity Model

Reviewed codebase

ITEM	VALUE
Repository	`blockventurechaincapital-crypto/bvcc-wallet-agent`
V5 commit	`838a70fabba4c9ab9ec26a13ea9059a63f75f150`
V5 `contracts-v5/src` tree	`64a5c674e5fdfd1e457daf59c8bf3bb813c89e62`
V4 commit	`a3e28f1a7c8e265d5b3630f98e4c8e77c6752eb5`
V4 `contracts/src` tree	`337e84f05fa11b948e3ad3221398ea4edc030f45`
Build	solc 0.8.36 · `via_ir` · `optimizer_runs = 50` · EVM `cancun` (frozen — CREATE2 addresses depend on it). V5 also drops the metadata hash, so the build reproduces the deployed bytecode byte for byte
Dependencies (V5)	OpenZeppelin Contracts `v5.6.1` (official tag) · forge-std `v1.16.1`, tests only
Tooling	Slither 0.11.3 · Foundry (forge 1.5.1; forge 1.8.6 in the V5 rounds) · viem 2.x

The V5 tree holds the core frozen on 2026-10-29 plus two validators added on 2026-11-02 without touching the core (Uniswap v3 positions and the wrapped native token). Earlier rounds reviewed the V1, V2 and V3 trees at their own commits; those findings and their remediations are in Appendix D and Appendix E.

In scope

V5 (`contracts-v5/src`):

- `BVCCSmartWalletV5.sol` — the passkey wallet: creation with a credential signature, owner execution and fees, signature expiry, guardian votes and recovery.
- `BVCCAgentWalletV5.sol` and the external library `BVCCAgentLogicV5.sol` — per-agent permissions, policies, pause and expiry, and the measured agent batch.
- `BVCCWalletFactoryV5.sol` — one factory contract, deployed twice, that creates non-upgradeable ERC-1167 clones by CREATE2.
- `BVCCValidatorRegistryV5.sol`, `BVCCHookRegistryV5.sol`, `IBVCCValidatorV5` and five validators: Universal Router, Uniswap v4 PositionManager, Aave v3 Pool, Uniswap v3 NonfungiblePositionManager and wrapped native (WETH, WBNB, WPOL).
- The deployment scripts, as they fix every address and the wiring between contracts.

V4 (`contracts/src`):

- `BVCCAgentWallet.sol` — agent permission logic (`executeAsAgent`, per-token & ETH caps, pause, expiry, the call-policy layer).
- `BVCCWallet.sol` — base smart account (execution, fees, guardian recovery).
- `BVCCAgentWalletFactory.sol` / `BVCCWalletFactory.sol` — CREATE2 factories.
- `BVCCValidatorRegistry.sol`, `BVCCUniversalRouterValidator.sol`, `BVCCPositionManagerValidator.sol`, `BVCCHookRegistry.sol` and the `IBVCCValidator` interface.

- The call-policy presets the interface registers, as configuration that determines on-chain behaviour.

Out of scope

- The third-party protocols the wallet interacts with (Uniswap, Aave, Permit2) — their own security is assumed, and the wallet is reviewed on the basis that they may behave adversarially at their interface.
- Bundler, paymaster and EntryPoint infrastructure.
- The web interface, except where it registers on-chain policy or is the only place a control exists (noted explicitly where that is the case).
- Economic and market risk, including price manipulation in pools the wallet trades against, except where it is the mechanism of a finding (BVCC-03).

Methodology

- **Manual review** of the agent execution path, permission accounting, recovery and the call-policy layer, in four passes across the four generations.
- **Static analysis** — Slither 0.11.3 over the tree at each round; results and the false-positive register in Appendix C.
- **Unit, fork and fuzz testing** — Foundry. Every finding has a regression test. The router validator is fuzzed over its whole command space against a model of the real router; Aave is exercised on a mainnet fork.
- **Exploit development** — findings are confirmed by a working proof of concept rather than by argument. BVCC-01 was reproduced against a wallet created from the *live* V2 factory on a fork.
- **On-chain testing** — the agent EOA drives `executeAsAgent` directly. Reverts are confirmed by simulation (`eth_call`) against live state; valid cases run as real transactions on Arbitrum Sepolia and Arbitrum One.
- **Deployment verification** — deployed bytecode, owner and registry wiring are read back from each network independently of the deployment script.
- **V5 review rounds** — five rounds between 2026-10-27 and 2026-10-29, each run with an AI analysis tool against the written specification and the code. Every finding was confirmed by a failing test, most of them on forks of the real networks, before it was fixed; the next round first checked the previous fixes.
- **V5 red team** — two sessions of AI agents attacking the code, one from inside (as agent, guardian, registry admin and owner) and one from outside with no role, plus 128,000 random action sequences checked against six core invariants. A separate AI-assisted review covered the two validators added after the freeze: 14 hypotheses, 13 adversarial tests, no findings.

Threat model

Stated explicitly because it changes how several findings should be read. A reader who assumes every issue requires stealing a key will misjudge the ones that require nothing at all.

ACTOR	WHAT IS ASSUMED
Owner	Trusted. Holds the passkey. Compromise of the passkey is out of scope — it is equivalent to owning the wallet, and guardian recovery exists for that case.
Agent	**Semi-trusted, and assumed hostile for the purpose of this review.** Its key lives on a server, in an environment variable or an env file, running unattended; and the agent may belong to a third-party operator the owner does not control. "The agent acts outside its mandate" is not an exotic precondition — it is the event the permission system exists to bound.

Guardian	Honest in the majority. Recovery requires two of three; the design accepts that two colluding guardians take the wallet if the owner does not cancel within 48 hours.
Third party	Untrusted, holds no credential of any kind, and can call every permissionless entry point — including the factories — in any order and on any network.
Protocol	Assumed correct in itself, but may behave adversarially at its interface: a pool may be attacker-controlled, a token may re-enter or return unexpected values.

> **Why "requires the agent's key" is not a mitigating factor.** The per-token caps, daily and lifetime budgets, expiry, pause and recipient whitelist exist precisely because the agent is not fully trusted. If a compromised or hostile agent key were treated as out of scope, the entire permission layer would have no purpose. Findings that hold *only* while the agent behaves are not limits; they are suggestions. This is why BVCC-01 is Critical: it converts "this agent may spend 0.003 ETH" into "this agent may take everything", which is a failure of the product's central promise rather than of one function. The preconditions are recorded per finding in §3 so the reader can weigh each one.

Severity model

Severity is **impact × likelihood**. Impact is the worst outcome for a wallet holder if the issue is exercised; likelihood weighs the preconditions an attacker must satisfy — key compromise, a race, a timelock, a specific wallet configuration — not how probable the attack is thought to be in the abstract.

SEVERITY	MEANING
CRITICAL	Loss of all wallet funds, or full takeover, with preconditions an attacker can realistically reach.
HIGH	Loss of funds beyond the configured limits, or takeover, but gated behind a race, a timelock, an owner action, or a specific configuration.
MEDIUM	Weakens a security control, degrades recovery, or denies service, without directly moving funds.
LOW	Violates a stated invariant with limited practical consequence.
INFORMATIONAL	No impact under current deployment; hardening, or a design decision recorded so it is not silently undone.

A finding's severity is stated as found, not as it stands after remediation. Status is tracked separately so that a fixed Critical is still visible as a Critical that was fixed.

Limitations of this review

Stated here rather than at the end, because it bears on how every finding below should be read. This is an **internal** review: the people who wrote the code also reviewed it. It has not been audited externally. Four of the findings came from a later pass over the V3 code with a different analysis tool, and each had survived three earlier rounds: changing the tool found what repeating the method did not. That pass was internal too — **no independent party has reviewed this code at all**. Absence of a finding in this document is therefore weak evidence of absence. §7 expands on what this review does and does not establish.

The V5 rounds share that limit and add one of their own: they were run with AI tools. That widened coverage, with seventeen issues found before deployment, but a tool reviews what it is pointed at, and four of the V5 rounds found a new path next to the previous fix. The red team and the fifth round found no theft path in the core; that is evidence about the paths they tried, not a proof.

3 Findings Register

Thirty findings: thirteen from the review of V4 and earlier (BVCC-01 to BVCC-13) and seventeen from the V5 rounds (BVCC-14 to BVCC-30), plus two closed in superseded generations (Appendix D). Severity is as found; status is as of 2026-11-11. For the V4-review findings the status column gives both generations, because V4 wallets are still in use.

ID	FINDING	REQUIRES	SEVERITY	STATUS
Review of V4 and earlier — 2026-06 → 2026-07				
`BVCC-01`	Cross-function reentrancy via `_currentAgent`	Agent key, or a hostile agent operator	CRITICAL	Fixed in V4
`BVCC-02`	Universal Router command mask accepts a bridge deposit as a swap	Agent key, with router access	CRITICAL	Fixed pre-deployment
`BVCC-03`	Token budgets do not bound protocol spending	Agent key + a standing allowance	HIGH	OPEN in V4 Closed in V5
`BVCC-04`	Guardian squatting through the permissionless factory	**None** — any third party	HIGH	Fixed in V4
`BVCC-05`	A completed recovery did not stop existing agents	A compromise already under way	HIGH	Fixed in V4
`BVCC-06`	Unauthenticated `credentialId` in the factory event	**None** — any third party	MEDIUM	Fixed in V4
`BVCC-07`	Credential left stale after a recovery	**None** — follows any recovery	MEDIUM	Fixed in V4
`BVCC-08`	Guardians could never be rotated	**None** — operational	MEDIUM	Fixed in V4
`BVCC-09`	A single guardian can stall a recovery indefinitely	One malicious guardian	MEDIUM	Accepted in V4 Closed in V5
`BVCC-10`	Pausing or revoking an agent leaves its allowances alive	A spender already approved	MEDIUM	OPEN in V4 — interface Closed in V5
`BVCC-11`	`approve` could carry ETH past the recipient whitelist	Agent key	LOW	Fixed in V4
`BVCC-12`	`increaseLiquidity` has no owner check in the v3 NFPM	Agent key; unreachable as deployed	INFORMATIONAL	Mitigated by exclusion in V4 Closed in V5

BVCC-13	Validator constructors accept a zero address	**None** — deployment error only	INFORMATIONAL	Accepted in V4 Closed in V5
V5 review — 2026-10-27 → 2026-11-02, all before or without a deployment				
BVCC-14	A failed balance read switched off an agent's token budget	Agent key + an allowed token whose balance cannot be read	HIGH	Fixed before deployment
BVCC-15	A protocol call could move an unlisted token packed in its calldata	Agent key + an old owner approval	HIGH	Fixed before deployment
BVCC-16	A protocol call could pay a recipient outside the agent's list	Agent key + a policy without a recipient anchor	HIGH	Fixed before deployment
BVCC-17	Aave withdraw-and-forward netted to zero against the budget	Agent key with Aave `withdraw`	HIGH	Fixed before deployment
BVCC-18	Paying the fee in a sender-taxed token exceeded the budget	Agent key + a sender-taxed allowed token	MEDIUM	Fixed before deployment
BVCC-19	A mutable validator could widen permissions without the 48-hour delay	Registry admin + a mutable validator	MEDIUM	Accepted — deployment rule
BVCC-20	A frozen hook was not checked when adding liquidity	Agent key + a frozen hook	MEDIUM	Fixed before deployment
BVCC-21	Liquidity moved into a foreign position for a dust charge	Agent key + a foreign position approving the wallet	HIGH	Fixed before deployment
BVCC-22	Liquidity netted into the wallet's own position approved to a third party	Agent key + an owner NFT approval	HIGH	Fixed before deployment
BVCC-23	An unapproved v4 hook moved an unlisted token through the Universal Router	Agent key + an old owner approval to the hook	HIGH	Fixed before deployment
BVCC-24	An agent withdrawal ran a frozen hook	Agent key + frozen hook + old owner approval	HIGH	Fixed before deployment
BVCC-25	An approved v4 hook runs code the wallet cannot bound	A malicious or compromised approved hook	HIGH	Accepted — trust decision
BVCC-26	An agent could drain a third party's position that approved the wallet	Agent key + a third-party NFT approval	HIGH	Fixed before deployment
BVCC-27	Agents could never burn a v4 position	**None** — normal use	MEDIUM	Fixed before deployment

BVCC-28	`pauseAgents` reverted when the wallet was already paused	**None** — owner operation	LOW	Fixed before deployment
BVCC-29	An allowed token that lies about its balance or moves another token	A malicious allowed token	MEDIUM	Accepted — trust assumption
BVCC-30	The wallet only measures tokens it can see or a validator declares	A validator that under-declares	LOW	Accepted — validator rule

By severity and status

SEVERITY	ALL	V4 REVIEW	V5 REVIEW
CRITICAL	2	2	0
HIGH	13	3	10
MEDIUM	10	5	5
LOW	3	1	2
INFORMATIONAL	2	2	0

STATUS	ON V5	ON V4 WALLETS
Fixed or closed	26	9 of 13
Open	0	2
Accepted as documented	4	2

Where the V5 identifiers come from

Each V5 round used its own numbering. They are renumbered here to continue the series; the original identifiers are kept so the round reports can be traced.

ROUND	DATE	ORIGINAL IDS	IN THIS REPORT	RESULT
V5 round 1	2026-10-27	`V5-H-01…04` · `V5-M-01…03`	BVCC-14 … BVCC-20	4 High, 3 Medium
V5 round 2	2026-10-28	`R2-H01`	BVCC-21	Round 1 fixes checked; 1 High
V5 round 3	2026-10-28	`R3-H01` · `R3-H02`	BVCC-22, BVCC-23	2 High
V5 round 4	2026-10-29	`R4-H01` · `R4-H02`	BVCC-24, BVCC-25	2 High (one accepted)
Red team	2026-10-29	`RT-01` · `RT-55` · `RT-30-A` · `RT-52/54` · `RT-65`	BVCC-26 … BVCC-30	No theft path in the core; 3 fixed, 2 accepted
V5 round 5	2026-10-29	—	—	Red-team fixes checked; no new findings; code frozen

New validators	2026-11-02	—			—	Uniswap v3 and wrapped native: 14 hypotheses, 13 tests, no findings

Reading the register. Severity is recorded as the issue was found, so a Critical that has been fixed still reads as Critical. The reader's question is usually "what was this codebase capable of", not only "what is left". On V5, nothing is open: twenty-six findings are closed in code and four are accepted trust assumptions that §6 spells out. On a V4 wallet, **BVCC-03** is still open and is the reason to migrate. The **Requires** column matters as much as the severity. Most V5 findings need an authorized agent plus something the owner did earlier, usually an old token or NFT approval, which the threat model in §2 treats as ordinary behaviour. One, BVCC-26, put a third party's position at risk rather than the owner's.

4 Findings in Detail

BVCC-01 to BVCC-13 come from the review of V4 and earlier; where V5 changed the outcome, a "Closed in V5" note says how. BVCC-14 to BVCC-30 come from the V5 rounds and follow them.

Each finding is stated with the severity it had when found, the preconditions an attacker needs, and the evidence that established it. Remediation and the verification of that remediation follow in §5.

The V3 source was put through a further internal pass with a different analysis tool, which raised four issues. All four were reproduced against the code rather than accepted on description; the tests live in `test/ThirdPartyClaims.t.sol`. All four hold, though two are narrower than stated and one is a property of ERC-20 rather than of this wallet. None of them had been found by the three internal rounds.

BVCC-01 CRITICAL Fixed in V4

Cross-function reentrancy via `_currentAgent`

LOCATION	`BVCCAgentWallet.sol` — `executeAsAgent` / `execute`
IMPACT	Total loss of wallet funds. Every configured limit is bypassed in a single transaction; a proof of concept moved **49.925 ETH** against a 0.003 ETH lifetime budget.
LIKELIHOOD	Medium. The agent address must acquire code after being authorized — an EIP-7702 delegation the agent signs with its own key, or a CREATE2 deployment to a pre-committed address. No owner action is required and nothing warns the owner.
AFFECTS	V1, V2 and V3 alike — confirmed by draining a wallet created from the *live* V2 factory on an Arbitrum One fork. Wallets that have not migrated to V4 remain exposed.

This result was initially dismissed on the grounds that an agent is an EOA and cannot be handed control mid-transaction. That reasoning does not hold, and the finding is confirmed.

Mechanism

`executeAsAgent` sets `_currentAgent = agent` around its call to `super.execute()`, and `_erc7821AuthorizedExecutor` grants the owner-level `execute()` path to any caller satisfying `caller == _currentAgent`. The parent `execute()` carries no reentrancy guard — `nonReentrant` sits on `executeAsAgent` only — and applies no agent limits, because the limit checks live in `executeAsAgent`. The EOA assumption rests on `AgentMustBeEOA` (`agent.code.length == 0`), which was enforced **only at authorization time** and never re-checked when the agent executed. An address can gain code afterwards: through an EIP-7702 delegation, signed with the agent's own key — the very key the threat model assumes an attacker may hold — or by deploying to a pre-computed CREATE2 address that was code-less when it was authorized.

> **Scope: V1, V2 and V3 alike.** This is not a V3 regression. The four ingredients — the flag, the executor override, the authorization-time-only EOA check and the unguarded parent `execute()` — are present verbatim in all three generations; only the line numbers moved. It was confirmed against deployed code rather than source: on an Arbitrum One fork, a wallet created from the **live V2 factory** (`0x8D9e2402…`) was drained by the same attack (`test/LegacyV2ReentrancyFork.t.sol`).

Reproduction

`test/AgentReentrancyPoC.t.sol`. An agent is authorized while code-less with caps of 0.001 / 0.002 / 0.003 ETH (per-tx / daily / lifetime). Code is then placed at the agent address, modelling

the delegation. The agent submits a batch whose only item is a **1 wei** transfer to itself — inside every cap — which hands it control; from there it calls `execute()` back into the wallet and moves **49.925 ETH** out of a 100 ETH balance. The 0.075 ETH difference is the wallet's own 0.15% fee, which confirms the drain travelled the owner-level path.

Blast radius

Every agent constraint is bypassed at once — per-transaction, daily, period and lifetime budgets, the recipient whitelist and the V3 call policies. It converts a bounded agent into owner-equivalent control of the wallet, a strictly larger hole than the exfiltration vector V3 was built to close. It does not affect the biometric owner's own path, and it does not let a third party who lacks the agent key do anything.

Remediation — two layers

Three candidate fixes were implemented and measured against three attack shapes rather than argued about. S1 sends 1 wei to the agent directly; S2 leaves the agent code-less on entry and has the batch itself deploy code to that address through a whitelisted helper (no EIP-7702 involved); S3 never names the agent at all, routing value to it through a whitelisted intermediary.

CANDIDATE	S1 DIRECT	S2 CREATE2 MID-BATCH	S3 INDIRECT	FACTORY SIZE
None (pre-fix)	drained	drained	drained	`24,283`
B — re-check `code.length`	blocked	**drained**	blocked	`24,294`
C — batch may not target the agent	blocked	blocked	**drained**	`24,312`
A — guard on external `execute()`	blocked	blocked	blocked	`24,317`
A + B (shipped)	blocked	blocked	blocked	`24,328`

The two rejected candidates failed for instructive reasons. **B alone** is another point-in-time check — the same class of assumption that caused the bug — and S2 walks straight past it by making the code appear *during* the batch. **C alone** only closes the one door the first proof-of-concept happened to use; S3 reaches the agent through a legitimate protocol instead.

What ships is both layers. **Layer 1** re-checks the EOA invariant on every execution, so it is enforced continuously rather than once. **Layer 2** overrides `execute()` in the agent wallet to reject any external entry while an agent batch is in flight (`ReentrantAgentExecute`); `executeAsAgent` reaches the parent through `super.execute()`, which resolves statically and therefore skips the override, so the legitimate path is untouched. Layer 2 is the one that does not care *how* or *when* the agent obtained code — it closes the window itself. Each layer has its own regression test, and layer 2 is proven independently through the S2 shape, where layer 1 passes cleanly and the drain is stopped anyway.

Cost: the agent factory grows 45 bytes to 24,328, leaving 248 bytes of EIP-170 headroom, and `executeAsAgent` case 3 rises from 64,758 to 64,903 gas — 145 gas, 0.2%, because the agent address is already warm as the transaction sender. One deliberate behavioural consequence: an agent that adopts an EIP-7702 delegation for unrelated reasons stops working until it is removed. That is the contract's own stated invariant, now enforced.

> **Deployed in V4 — but only for wallets created on V4.** The fix changes the wallet bytecode and therefore the CREATE2 addresses, so it cannot reach a wallet already deployed; `Create2Consistency.t.sol` failing against the old constants was the mechanical proof of that. V4 factories are live on all six networks, **and every wallet still on V1, V2 or**

V3 runs vulnerable bytecode until its owner migrates. The mitigation for those wallets, without redeploying: authorize agents with a non-empty `allowedRecipients` that does not contain the agent's own address, which removes the batch's ability to hand control back to it (`test_RecipientWhitelist_BlocksTheEntry`). Wallets left on the permissive default — an empty recipient list, meaning any destination — are the exposed configuration. Pausing or revoking the agent also closes it immediately.

Among the Low results, 20 `calls-loop` and 4 `timestamp` are inherent to batch execution and to day-indexed limits; `return-bomb` on `_tryBalanceOf` is bounded by the same `PROBE_GAS_CAP` introduced in V2, which also caps how much return data a hostile token can afford to produce. One result is a genuine, if inert, gap:

Accepted nit — no zero-check in the validator constructors. Neither `BVCCUniversalRouterValidator` nor `BVCCPositionManagerValidator` rejects a zero address for the protocol it binds to, unlike the factory constructors hardened in V1 (`ZeroOwner`). A validator bound to the zero address would deny every call — fail-closed, so the deployed behaviour is safe — and the six deployed instances carry the correct constructor arguments, recorded with their codehashes in `contracts/deployments/` and verified on the explorers. Adding the check would change the bytecode, and therefore the CREATE2 addresses, forcing a redeploy and a fresh 48-hour governance cycle per network. It is recorded here for the next bytecode revision rather than acted on now.

The Informational results are the same classes as in the V1 and V2 runs: inline assembly, low-level calls, naming conventions on immutables, pragma spread across the OpenZeppelin tree, and the `_feeNumerator()` "dead code" report — a known false positive: the function is reached through virtual dispatch and sets the 0.15% agent fee.

Re-run after the V4 fixes: 53 results — 3 High, 27 Low, 23 Informational. The three High results are unchanged, and all three are now false positives: the two `uninitialized-state` reports are mappings, and `reentrancy-eth` still sees the flag written after the external call even though BVCC-01 closed the window — it now lists `execute()` among the reachable functions, which is precisely where the guard sits. A second `dead-code` report appeared for `_afterRecovery()`, the recovery hook of BVCC-05: the same false positive as `_feeNumerator()`, an empty virtual overridden in the child. Neither is to be removed. Nothing new was introduced by the fixes. **Static analysis is a pattern detector, not a verdict**: this count did not move while the reentrancy was exploitable for three generations, and it did not move when the fix closed it.

BVCC-02 CRITICAL Fixed pre-deployment

Universal Router command mask accepts a bridge deposit as a swap

LOCATION	`BVCCUniversalRouterValidator.sol` — command decoding
IMPACT	Critical. The router would bridge the wallet's funds to an attacker on another chain while the validator classified the call as an ordinary swap — the exfiltration class the validator exists to prevent.
LIKELIHOOD	Medium — a compromised agent key with Universal Router access, and nothing else.
NOTE	Found in review **before** the validator was registered on any network, so it was never exploitable in production. Recorded because the near-miss is the point: the validator was written to stop exactly this and would have permitted it.

Critical bug caught during review — the command mask. The first version of the validator masked command bytes with `& 0x3f`. The real router masks with `& 0x7f` (bit `0x80` is `FLAG_ALLOW_REVERT`), which means `0x40` is **not** an alias of `0x00` — it is `ACROSS_V4_DEPOSIT_V3`, a cross-chain bridge deposit. Under the wrong mask the validator would have accepted it as a v3 swap while the router bridged the wallet's funds to an attacker on another chain: precisely the exfiltration class V3 exists to prevent. The fix replaced masking with exact matching, bound the validator to a single router and rejected empty command strings. The behaviour is now fuzzed across all 256 command bytes against a model of the router, and was re-confirmed on mainnet after deployment (see Appendix B).

BVCC-03 HIGH OPEN

Token budgets do not bound protocol spending

LOCATION	`BVCCAgentWallet.sol` — `_validateExecutionItem`, case 3
IMPACT	Critical where it applies: loss of the entire standing allowance, irrespective of the token budget configured for the agent.
LIKELIHOOD	Medium-low. Requires both a compromised agent key *and* an allowance the owner has already granted to a protocol the agent can reach. A wallet with no standing allowance is not exposed.
NOTE	The only finding of the V4 review that V4 does not close. **Closed in V5** by measuring outflows; see the note at the end of this finding. V4 wallets remain exposed.

The per-token counters accumulate only for the ERC-20 `transfer` and `approve` selectors: `_checkTokenWhitelistAndAmount` is reached from those two branches and from nowhere else. A case-3 protocol call therefore spends tokens without touching `tokenMaxAmounts`, the daily limit or the lifetime budget — and without comparing the asset inside its calldata against `allowedTokens`. This is deliberate as far as it goes: the model charges the budget at the `approve` step, which is what the Aave fork suite asserts, so in the ordinary approve-then-trade flow the budget is consumed.

The gap is **pre-existing allowances**, and the common case is not exotic: an owner who has granted a router or Permit2 a large allowance from the interface — standard practice — leaves the agent able to move that token through case 3 indefinitely without consuming a unit of its token budget. Reproduced in `test_Claim2_PreExistingAllowanceEscapesTokenBudgets`: with all three token caps set to 10, the agent moves 500 through a whitelisted router and both counters stay at zero. Permit2's own `approve(address,address,uint160,uint48)` is likewise a different selector and is not counted.

Severity correction. An earlier draft of this section described the loss as trading beyond the intended volume, on the grounds that call policies pin every recipient to the wallet. That understates it. The pin guarantees *where the output lands*, not *what it is worth*: a compromised agent can swap a valuable asset into a worthless one through a pool it controls, and the worthless token dutifully comes home while the value stays in the attacker's liquidity. Nothing in the current design objects — the Universal Router validator contains no reference to amounts or prices at all, and the SwapRouter02 policies pin the recipient and nothing else, so a swap with `amountOutMinimum = 1` passes every check. Where a standing allowance exists, this is effectively a full drain of the tokens it covers, not slippage.

Token selection narrows this but does not close it. Choosing which tokens an agent may touch does govern `transfer` and `approve`, so an unselected token with no standing allowance cannot be moved: the agent cannot approve it either. Two holes remain. A token that already carries an allowance is reachable regardless of the selection. And Aave's `borrow` and `withdraw` consume no allowance at all, so there is nothing to zero: an agent holding an Aave policy can take on debt in an asset the owner never selected, against the owner's collateral, with the liquidation risk that implies. The funds land in the wallet — this is damage rather than theft — but it is damage the token selection was expected to prevent.

Remediation plan

Full in-wallet accounting would require deep validators to decode and return the `(token, maxSpend)` pairs they see, with the wallet accumulating them through `_applyTokenSpend` and counting `amountInMaximum` for exact-output routes. That is a change to every validator plus the wallet, and the agent factory has 101 bytes of EIP-170 headroom. It is therefore staged rather than attempted now.

STAGE	MEASURE	COST
Now interface only	Rename the limit so it stops implying a global cap — it governs direct transfers and approvals, not DeFi volume	None
	At authorization, read the allowances of every relevant token towards every protocol the agent's policies can reach, and require a signed revocation of any that is non-zero	Frontend + one signed batch
	Drive agent operations as atomic batches: `approve(exact)` → protocol call → `approve(0)`, so the first leg consumes budget and nothing outlives the batch	Frontend; works on today's contract
	For Permit2, check both layers — the ERC-20 allowance to Permit2 and Permit2's internal allowance to the router — and use exact amounts with short expirations	Frontend
	An owner-facing emergency screen that revokes all allowances, and modest balances on wallets that carry agents	Frontend + operational
Next validator layer	Price control in the swap validators: reject a route whose `amountOutMinimum` is implausible against an oracle or TWAP. A validator is a `view` function, so it can read one. **This is the measure that closes the rigged-pool drain.**	New validators; 48 h governance per network
	An Aave validator that checks the `asset` argument against the wallet's selected tokens, so the selection governs lending too	New validator; same cycle
Later	An agent vault the agent must route through, so the amount handed to it is a physical ceiling and routers leave `allowedProtocols` entirely	New contract + integration
	In-wallet accounting, once a factory redesign frees the bytecode for it	Requires V5-scale work

> **How the severity comes down.** The rating is driven by two things: the loss has no ceiling, and the precondition is ordinary user behaviour. Note what happens when no standing allowance exists — the agent must `approve` before it can spend, and that approve *is* counted against the per-token budget and gated by the token and recipient lists. The budget bounds everything again. The finding is therefore not "the budget does not work" but "the budget does not work *while a standing allowance exists*", and that condition is attackable without touching the contracts.

STATE	WHAT IS BOUNDED	SEVERITY
Today	Nothing, where an allowance exists	**HIGH**
+ allowance hygiene (zero at authorization, exact atomic approvals, short Permit2 expirations)	Cumulative spend, via the token budget	MEDIUM
+ price control in the swap validators	Per-operation loss as well	LOW

Hygiene alone stops at medium because it is a snapshot: an owner can grant an allowance from any dApp afterwards. It lowers the likelihood; only the validator lowers the impact. Two caveats: Aave's `borrow` and `withdraw` consume no allowance, so hygiene does not reach them until an Aave validator checks the `asset`; and a price bound caps each operation, not the total — bounding the cumulative case-3 spend needs the in-wallet accounting the bytecode cannot currently hold.

What the interface measures cannot do. Blocking unlimited approvals while an agent is active is not enforceable from the wallet's own frontend: an owner can grant an allowance from any dApp, and nothing on-chain prevents it. The authorization-time check is therefore a snapshot that closes the common case — the forgotten standing allowance — not an invariant. Only the validator layer makes it one.

Closed in V5. V5 took the "in-wallet accounting" row of the plan above and built it into the core, by measuring rather than by decoding. Each agent batch reads the balance of every allowed token, every token found in the calldata and every token a validator declares, before and after it runs. Whatever left the wallet is charged against the agent's per-batch, daily and total limits, with the 0.15% fee included. A standing allowance no longer matters: a swap through an approval the owner granted earlier is charged like any other spend, as tested on an Arbitrum fork against the real Uniswap contracts. A swap into a worthless token is still charged at the full value that left, so the loss is bounded by the agent's budget. V5 does not add a price check. Aave is covered too: `borrow` is charged against the budget, the asset must be on the agent's list, and borrowing, withdrawing or changing collateral is refused if the health factor would fall below 1.2. Withdrawals are charged as they come in (BVCC-17). The new mechanism brought its own findings, BVCC-14 to BVCC-30, all handled before deployment.

BVCC-04 HIGH Fixed in V4

Guardian squatting through the permissionless factory

LOCATION	`BVCCWalletFactory.sol` / `BVCCAgentWalletFactory.sol` — `createWallet`; `BVCCWallet.sol` — `setGuardians`
IMPACT	Critical. Full takeover: the squatter's guardians rotate the owner once the recovery timelock elapses.
LIKELIHOOD	Low. The attacker must deploy the victim's address first on a network the victim has not used, the victim must then fund it, and the 48-hour window must pass without the owner cancelling. Detectable by reading `guardians()` before funding.
REPORTED BY	Later internal pass, different tooling

A wallet's CREATE2 address derives from the passkey public key alone; the guardians are not part of it. `createWallet` is permissionless and sets the guardians in the same call, and

`setGuardians` is one-shot. Whoever deploys an address first therefore chooses, permanently, the three parties who can rotate its owner. If the wallet already exists the factory returns it unchanged, so a caller arriving second is handed the squatted wallet with no error and no signal.

The full takeover was reproduced end to end (`test_Claim1_GuardianSquattingTakesOverTheWallet`): an attacker deploys the victim's deterministic address with its own guardians; the victim's app then "creates" the wallet and receives the same address back; two of the attacker's guardians open a recovery, wait out the 48-hour timelock and rotate the signer. The wallet and its balance are theirs.

The multichain design amplifies this precisely because it is deterministic: the passkey public key becomes visible on-chain the moment the user deploys anywhere, and the same address is claimable on every other network. The exposure is therefore **networks where a user has not yet deployed**, not wallets already in use — once correct guardians are set they cannot be replaced. The 48-hour timelock plus `cancelRecovery` is a real defence, but only on a chain the owner is watching, which by definition excludes the chains they have not deployed to.

> **One part of the reviewer's recommendation does not hold.** Rejecting duplicate guardians on-chain is hygiene, not a hole: approvals are tracked per address, so a duplicated guardian cannot approve twice and cannot reach the 2-of-3 threshold alone (`test_Claim1_DuplicateGuardianCannotSelfApprove`). Duplicates degrade the scheme to 2-of-2; they do not enable a solo takeover.

Remediation, applied in source. The factory no longer chooses guardians: it deploys the wallet with none, and `setGuardians` is gated behind `msg.sender == address(this)` — the same self-call pattern `authorizeAgent` and `setCallPolicy` already use — so only a passkey-signed operation can set them. Duplicates are now rejected as well. A squatter is left with an empty shell it cannot configure, and is in effect paying the victim's deployment gas. Guardians are read nowhere except the recovery path, so a wallet without them operates normally; only recovery is unavailable until the owner signs. Seven regression tests cover it, including the owner's real path — a `execute()` batch whose item targets the wallet itself — and the fact that an authorized agent cannot reach it either, because `executeAsAgent` refuses any item targeting the wallet.

BVCC-05 HIGH Fixed in V4

A completed recovery did not stop existing agents

LOCATION	`BVCCWallet.sol` — `executeRecovery`; `BVCCAgentWallet.sol` — `_afterRecovery`
IMPACT	High. The attacker's agent keeps spending after the owner believes the attacker has been locked out — and compromise is precisely why an owner reaches for recovery.
LIKELIHOOD	Medium — applies to any recovery performed in response to a compromise, which is the main reason the mechanism exists.

Recovery did not revoke agents. FIXED IN V4 `executeRecovery` rotated the signer and touched nothing else, so an agent authorized before the recovery kept spending after it, budget intact. That matters because compromise is precisely why an owner reaches for recovery: an owner who recovers believing they have locked the attacker out had not revoked the attacker's agent. Recovery now pauses agents through an `_afterRecovery` hook. Pausing rather than revoking is deliberate — constant gas whatever the agent list looks like, reversible, and it preserves each agent's configuration for the new owner to judge rather than forcing them to

rebuild it. The guard is `if (!paused()) _pause()`: without it, a wallet whose agents were already paused could not have completed a recovery at all.

BVCC-06 MEDIUM Fixed in V4

Unauthenticated `credentialId` in the factory event

LOCATION	Factory `WalletCreated` event; `BVCCWallet.sol` — `_setCredential` / `setCredentialId`
IMPACT	Medium. A squatter could publish a bogus credential for someone else's address, and after a recovery the on-chain credential still pointed at the passkey that had just been replaced — the interface would offer the wrong key.
LIKELIHOOD	Medium — the first follows from BVCC-04; the second from any completed recovery.
REPORTED BY	Later internal pass, different tooling — follow-up review

Two related findings surfaced while fixing it. The `credentialId` travelled through the factory as an unauthenticated event field, so a squatter could publish a bogus one for someone else's address. It cannot cost funds — signatures are verified against the stored public key — but a client that trusts it pre-selects a credential the owner does not hold, and the passkey prompt fails. It has been moved out of the factory entirely: the wallet now announces it itself, in the same passkey-signed call that sets the guardians, through `CredentialSet(bytes32 indexed credentialHash, bytes credentialId)`. The hash is indexed so a client holding a passkey `rawId` can find the wallet it belongs to.

BVCC-07 MEDIUM Fixed in V4

Credential left stale after a recovery

LOCATION	`BVCCWallet.sol` — `executeRecovery` / `setCredentialId`
IMPACT	Medium. After a guardian recovery the on-chain credential still pointed at the passkey the owner no longer held, so a client trusting it prompts for the wrong key and the recovered wallet appears unusable.
LIKELIHOOD	Certain after any completed recovery — not an attack, a broken end state for the mechanism that exists to rescue the wallet.
REPORTED BY	Later internal pass, different tooling — follow-up review

Discovered while fixing BVCC-04. `executeRecovery` rotates the signer but left the credential untouched, so after a guardian recovery the on-chain credential pointed at a passkey the owner no longer held; `setCredentialId`, self-call gated, closes that.

BVCC-08 MEDIUM Fixed in V4

Guardians could never be rotated

LOCATION	`BVCCWallet.sol` — `setGuardians`
IMPACT	Medium. A guardian whose key was later lost or compromised was permanent for the life of the wallet; the only remedy was to create a new wallet and move the funds.
LIKELIHOOD	Low as an attack, certain as an operational failure over a long enough horizon.

Guardians could never be rotated. FIXED IN V4 `setGuardians` was one-shot, so a guardian whose key was later lost or compromised was permanent for the life of the wallet and the only remedy was to create a new wallet and move the funds. The set is now replaceable by the

owner. It remains a self-call, so the squatting of BVCC-04 stays closed, and rotation is refused while a recovery is in flight (`RecoveryActive`) — otherwise a stolen passkey could swap the guardians out from under a recovery already under way. The owner cancels first, then rotates.

BVCC-09 MEDIUM Accepted

A single guardian can stall a recovery indefinitely

LOCATION	`BVCCWallet.sol` — `initiateRecovery`
IMPACT	Medium. Denial of service against the rescue mechanism. No funds move and the owner is never rotated.
LIKELIHOOD	Medium — requires one malicious or compromised guardian willing to front-run each second approval.
NOTE	Accepted as a documented limitation: it is a race rather than a lock, and once the honest pair lands two approvals nobody can reset it.

A single guardian can stall a recovery indefinitely. OPEN — ACCEPTED `initiateRecovery` may be called whenever approvals are below two, and it overwrites the pending key and zeroes the timelock. One malicious guardian therefore only has to front-run each second approval to keep the honest pair from ever reaching the threshold. This is denial of service against the rescue mechanism, not theft: the owner never rotates. It is also a race rather than a lock — a companion test shows that once the honest pair lands two approvals, no one can reset it. Accepted as a documented limitation.

> **The trust model, stated plainly.** Two colluding guardians take the wallet if the owner does not cancel inside the 48-hour window — that is what makes recovery work, and it is tested as such. It means the owner must be watching every network the wallet exists on, which is the same operational requirement the guardian squatting of BVCC-04 exposed from a different direction.

> **Closed in V5.** Recovery no longer has an "initiate" step that anyone can overwrite. Each guardian votes for one concrete proposal (new key and credential) and controls only its own vote. Two matching votes arm the proposal and start the 48 hours; a third guardian cannot reset or remove the others' votes, and a proposal stays armed while it holds two votes. A hostile guardian can at most delay one cycle by voting and withdrawing. 128,000 random guardian actions confirmed the invariants: armed exactly when two votes match, no guardian changes a proposal it does not hold up, and the key never changes before 48 hours. The owner can still cancel at any time with the passkey.

BVCC-10 MEDIUM OPEN — interface

Pausing or revoking an agent leaves its allowances alive

LOCATION	ERC-20 allowance semantics; interface revocation flow
IMPACT	Medium. An owner who pauses a compromised agent has not stopped the spender it already approved. The allowance survives in the token contract.
LIKELIHOOD	Medium — every revocation performed under the reasonable assumption that it is enough.
NOTE	Inherent to ERC-20: nothing the wallet does can retract an allowance held by the token. The remedy is an interface that revokes explicitly — see §6.

An allowance lives in the token contract, so nothing the wallet does can retract it. Reproduced in `test_Claim3_AllowanceSurvivesPauseAndRevoke`: the agent approves within its cap, the owner pauses *and* revokes, and the spender still pulls the full amount. The reviewer correctly notes the amount was charged to the budget at approval time, so no limit is evaded. This is a property of every smart account, not a defect of this one, and it matters for incident response rather than for the permission model: after revoking an agent, an owner should also zero the allowances it was granted. An explicit warning at pause time and an owner-facing allowance revocation screen cover it.

> **Closed in V5.** An agent can no longer leave an allowance behind. At the end of every agent batch, each approval the agent made is set back to its previous value (through zero first, for tokens such as USDT), so the owner's own approvals are untouched. An agent `approve` may only name a protocol on the agent's list, never a plain recipient. Allowances the *owner* grants still live in the token contracts; the owner manages them, and the agent cannot use them beyond its budget.

BVCC-11 LOW Fixed in V4

`approve` could carry ETH past the recipient whitelist

LOCATION	`BVCCAgentWallet.sol` — `_validateExecutionItem`
IMPACT	Low. Violates the contract's own stated invariant that token calls carry no value; the practical consequence is bounded by the ETH budget, which still applies.
LIKELIHOOD	Low — requires a compromised agent key and a token that accepts value-bearing calls.
REPORTED BY	Later internal pass, different tooling

An execution carrying both a positive `value` and `approve` calldata is classified by the agent validator as a token operation — so the recipient check is applied to the *spender* — while the parent `execute()` falls through to case 3 and forwards the ETH to the token address. The ETH caps still apply, since `value` is accumulated unconditionally; what is skipped is the destination whitelist. Reproduced in `test_Claim4_ApproveCarriesEthPastRecipientWhitelist`, where a plain ETH send to the same address reverts `RecipientNotAllowed` and the same ETH riding on an `approve` arrives.

Severity is limited by a detail the reviewer does not mention: the target's `approve` must be `payable` for the call to succeed at all. Standard ERC-20s are not, so the attempt simply reverts; the test needs a purpose-built payable token to demonstrate the path. Exploitation therefore requires the owner to have whitelisted an unusual or hostile token. A related piece of untidiness sits next to it: a `transfer` carrying `value` is executed by the parent without forwarding the ETH, yet the agent still charges that value to its ETH budget. Both are now refused: a token call carrying value reverts `TokenCallWithValue`, verified by a regression test that also confirms an ordinary value-free `approve` still works.

BVCC-12 INFORMATIONAL Mitigated by exclusion

`increaseLiquidity` has no owner check in the v3 position manager

LOCATION	Uniswap v3 `NonfungiblePositionManager`; call-policy presets
IMPACT	High if it were whitelisted: unlike `decreaseLiquidity`, `collect` and `burn`, the position manager performs no owner check on `increaseLiquidity`. An agent could pull the

	wallet's approved tokens into a position owned by the attacker — a recipient-whitelist bypass that cannot be pinned, because word 0 is the `tokenId`, not an address.
LIKELIHOOD	Not reachable as deployed — the selector is deliberately absent from every policy preset.
NOTE	Recorded so the exclusion is not silently undone by someone adding the selector for convenience. To add liquidity, the agent mints a fresh position with the recipient pinned to the wallet. Re-enabling it needs a deep validator that checks `ownerOf(tokenId) == wallet`. **Closed in V5:** that is what the V5 Uniswap v3 validator does. `increaseLiquidity` is allowed only on a position the wallet owns and that has no individual approval.

BVCC-13 `INFORMATIONAL` `Accepted`

Validator constructors accept a zero address

LOCATION	`BVCCUniversalRouterValidator.sol`, `BVCCPositionManagerValidator.sol` — constructors
IMPACT	None as deployed. A validator constructed with a zero router or position manager would match nothing and deny every call — a failure in the safe direction.
LIKELIHOOD	Deployment-time error only; not attacker-controlled.
NOTE	Every deployed instance was read back on all six networks and both immutables are non-zero. Accepted rather than fixed: adding the check would require redeploying and re-registering validators that are demonstrably correct. **Closed in V5:** every V5 validator constructor reverts on a zero address.

V5 review — BVCC-14 to BVCC-30

The findings below come from the five V5 rounds and the red-team exercise. None of them reached a deployed contract: V5 was deployed after the code was frozen on 2026-10-29. Each was confirmed by a test that failed on the code as it then stood, most of them on a fork of a real network with the deployed Uniswap and Aave contracts. Those tests now pass as regression tests, and they must keep failing on the old code. The V5 guarantees they refer to (G-1 to G-7) are listed in Appendix A.

BVCC-14 `HIGH` `Fixed before deployment`

A failed balance read switched off an agent's token budget

LOCATION	`BVCCAgentLogicV5.sol` — `prepare` / `settle` (measurement)
IMPACT	An allowed token whose `balanceOf` reverts, or needs more than the 100,000-gas probe cap, left the wallet with no charge at all. Repeated sends could empty it.
LIKELIHOOD	Low: needs an unusual token, or one whose implementation changes. Standard ERC-20s are not affected.
SOURCE	V5 round 1, `V5-H-01`. Breaks G-2.

The pre-batch measurement stored whether each balance read succeeded and went on regardless. At settlement, a token whose read had failed was treated as having no outflow. The proof of concept allowed one unit of a token whose `balanceOf` reverts and sent ten: the transfer went through and the budget recorded nothing.

Fix. A failed read now reverts the whole batch with `MeasurementFailed` before anything runs (`test_Measure_UnreadableBalanceReverts`). The cost is accepted and documented: such a token blocks that agent's batches until the owner removes it from the agent's list.

BVCC-15 HIGH Fixed before deployment

A protocol call could move an unlisted token packed in its calldata

LOCATION	`BVCCAgentLogicV5.sol` — protocol branch and calldata scan
IMPACT	Using an approval the owner had granted earlier, an allowed protocol could pull a token the agent was never allowed to touch, with no budget check.
LIKELIHOOD	Medium: needs an old owner approval and a protocol selector the owner enabled without a validator.
SOURCE	V5 round 1, `V5-H-02`. Breaks G-2 and G-3.

The wallet finds the tokens to measure by scanning calldata for whole 32-byte address words. A token packed inside a `bytes` path, as routers encode them, is invisible to that scan. In the first V5 draft, a protocol selector could be allowed without a validator, so nothing else declared the token. The proof of concept used a puller contract with a packed path and moved 100 units of a token outside the agent's list.

Fix. Every protocol call by an agent now needs a validator: a policy without the `DEEP` bit reverts with `CallNeedsValidator`, whatever the selector. The old list of "dangerous selectors" is gone. The validator is responsible for declaring every token the call can move (see BVCC-30). As a consequence, SwapRouter02 is no longer available to agents; their swaps go through the Universal Router. Regression: `test_Policy_EveryProtocolCallNeedsValidator`, plus `test_Fork_UR_NotAllowedTokenRejected` on the real router.

BVCC-16 HIGH Fixed before deployment

A protocol call could pay a recipient outside the agent's list

LOCATION	`BVCCAgentLogicV5.sol` — protocol branch
IMPACT	An allowed protocol function that takes a destination argument could pay an address the agent chose, outside its recipient list. The amount was charged, but the destination was not checked.
LIKELIHOOD	Medium: needs a protocol policy the owner enabled without a recipient anchor.
SOURCE	V5 round 1, `V5-H-03`, rated "High, conditional". Breaks G-4.

Direct sends checked the recipient list; protocol calls checked only the anchors the owner had set on the policy, and the recipient anchor was optional. The finding was conditional on how G-4 was read, so the team settled it: G-4 applies to every payout an agent can cause.

Fix. The same as BVCC-15. With a validator required on every protocol call, each validator pins every payout recipient of the functions it allows (in practice, the wallet itself). Regression: `test_Policy_EveryProtocolCallNeedsValidator` and `test_Fork_UR_ForeignRecipientRejected`.

BVCC-17 HIGH Fixed before deployment

Aave withdraw-and-forward netted to zero against the budget

LOCATION	`BVCCAaveV3ValidatorV5.sol` — `withdraw`; `BVCCAgentLogicV5.sol` — settlement

IMPACT	An agent could turn Aave collateral into cash and send it out in one batch while its budget recorded nothing. Reproduced against the real Aave v3 Pool: 99 USDC out with a 1 USDC limit.
LIKELIHOOD	Medium: needs an agent with Aave `withdraw` and collateral in the wallet. Nothing else.
SOURCE	V5 round 1, `V5-H-04`. Breaks G-2.

V5 charges the net outflow of each measured token over the batch. The aToken was not measured, so withdrawing 99 USDC into the wallet and then sending 99 USDC out left the USDC balance where it started: net outflow zero, charge zero, fee zero, and the collateral gone. The fork test at Arbitrum block 518,809,446 asserted exactly that.

Fix. Charged inflows. The validator interface gained a fifth return value, `chargeInflow`. When a validator sets it, the wallet reads the declared tokens before and after that one call and adds what came in to the agent's costs. Aave `withdraw` and every Uniswap v4 PositionManager call use it, so taking money out of a position the wallet does not measure is charged at 0.15% like any other spend. On the real Pool, withdrawing 99 USDC and forwarding it with a limit of 10 now reverts, and withdrawing 5 charges 5.0075 (`test_Fork_Aave_WithdrawIsChargedAndCapped`).

BVCC-18 MEDIUM Fixed before deployment

Paying the fee in a sender-taxed token exceeded the budget

LOCATION	`BVCCAgentLogicV5.sol` — fee payment
IMPACT	With a token that charges the sender a tax on every transfer, the fee transfer itself cost more than what was charged, so the wallet lost slightly more than the agent's limit.
LIKELIHOOD	Low: needs a sender-taxed token on the agent's list. The excess is small and token-dependent.
SOURCE	V5 round 1, `V5-M-01`. Breaks G-2 by a small amount.

The charge was recorded before the fee was paid, and the balance was not read again afterwards. With a 10% sender tax and a limit of exactly 110.165 units, a 100-unit send cost the wallet 110.1815.

Fix. After paying each fee, the wallet reads the balance again and reverts if it fell by more than the fee (`test_Fee_SenderTaxedTokenReverts`). Agents therefore cannot move sender-taxed tokens at all; tokens that tax the receiver still work. This is listed among the V5 limits in §6.

BVCC-19 MEDIUM Accepted — deployment rule

A mutable validator could widen permissions without the 48-hour delay

LOCATION	`BVCCValidatorRegistryV5.sol` — registration
IMPACT	The registry delays a new validator *address* by 48 hours, but calls whatever code is at that address. A validator with its own admin, or behind a proxy, could start saying yes at once.
LIKELIHOOD	Low as deployed: needs the registry admin to register a mutable validator. None of the five deployed validators has state or an admin.
SOURCE	V5 round 1, `V5-M-02`. Would break G-6.

The registry cannot tell whether the code it points to can change its mind. A code-hash check would not catch a state change or a proxy upgrade, so there is no one-line contract fix.

Decision. G-6 now carries a deployment rule: only immutable, stateless validators are registered. The five deployed validators take every parameter in the constructor as an immutable and hold no storage. Their source is verified on every explorer, so anyone can check that before the 48 hours run out. Registering a validator that breaks this rule would weaken G-6, which is why it is listed as an accepted trust assumption rather than fixed.

BVCC-20 MEDIUM Fixed before deployment

A frozen hook was not checked when adding liquidity

LOCATION	`BVCCPositionManagerValidatorV5.sol` — `INCREASE_LIQUIDITY`
IMPACT	After governance froze a Uniswap v4 hook, an agent could still add liquidity to an existing position in that hook's pool.
LIKELIHOOD	Low: needs a hook that was approved and later frozen, and a position in its pool.
SOURCE	V5 round 1, `V5-M-03`.

The validator checked the hook on `MINT_POSITION` but returned yes for `INCREASE_LIQUIDITY` without looking up the position's pool.

Fix. The validator reads the position's pool key from the PositionManager and requires its hook to be none or approved (`test_PM_IncreaseChecksHook`). The same branch later gained the owner checks of BVCC-21 and BVCC-22.

BVCC-21 HIGH Fixed before deployment

Liquidity moved into a foreign position for a dust charge

LOCATION	`BVCCPositionManagerValidatorV5.sol` — `INCREASE_LIQUIDITY`
IMPACT	An agent could empty a wallet position into a position it owned. Reproduced on Arbitrum with the deployed PositionManager: liquidity of 1e12 moved over while the agent was charged at most two base units of each token.
LIKELIHOOD	Medium: needs an agent managing v4 liquidity and a wallet position. The agent arranges the rest itself.
SOURCE	V5 round 2, `R2-H01`. Breaks G-2 and G-4.

The Uniswap v4 PositionManager accepts actions from a position's owner *or an approved operator*. An agent can mint its own position and approve the wallet as operator. The validator checked only the hook on `INCREASE_LIQUIDITY`, so a single `modifyLiquidities` call could decrease the wallet's position and increase the agent's in the same pool and range. The PositionManager nets the two deltas internally, so almost no ERC-20 left the wallet and the charged inflow (BVCC-17) saw nothing to charge. The specification had assumed "Uniswap v4 already requires the owner"; that note was wrong.

Fix. `INCREASE_LIQUIDITY` requires `ownerOf(tokenId) == wallet`. Regression on the Arbitrum fork: `test_Fork_PM_ForeignOwnedNFTRejected` and `test_Fork_PM_LiquidityMoveToForeignNFTRejected`.

BVCC-22 HIGH Fixed before deployment

Liquidity netted into the wallet's own position approved to a third party

LOCATION	`BVCCPositionManagerValidatorV5.sol` — action combinations

IMPACT	Loss of a wallet position: an agent moved its liquidity into a second wallet position that the owner had approved to someone else, who then took that position.
LIKELIHOOD	Medium-low: needs an individual NFT approval the owner granted earlier on one of the wallet's positions.
SOURCE	V5 round 3, `R3-H01`. Breaks G-2 and G-4.

The fix for BVCC-21 checks that the target position belongs to the wallet. It does not stop the target being approved to a third party. In the fork test the owner holds two positions in the same pool and range and has approved only the small one to a third party. The agent decreases the large one and increases the small one in one call; the third party then transfers the small position out with the approval it already had.

Fix. Two rules. One call to the PositionManager can no longer both remove liquidity (`DECREASE`/`BURN`) and add it (`MINT`/`INCREASE`), in any order, because that is what lets liquidity change position with no token movement to charge. And `INCREASE_LIQUIDITY` requires the position to have no individual approval (`getApproved == 0`). Moving liquidity between positions now takes two calls, and what comes out pays 0.15% as an inflow. Regression: the three tests in `AuditPositionManagerForkV5.t.sol`.

BVCC-23 HIGH Fixed before deployment

An unapproved v4 hook moved an unlisted token through the Universal Router

LOCATION	`BVCCUniversalRouterValidatorV5.sol` — `V4_SWAP` / `SWAP_EXACT_IN`
IMPACT	An agent with no allowed tokens at all routed a swap through a pool with a hook of its choosing; the hook spent an old owner approval and took 1e18 units of an unlisted token.
LIKELIHOOD	Medium-low: needs an old owner approval to the hook address and Universal Router access for the agent.
SOURCE	V5 round 3, `R3-H02`. Breaks G-2, G-3 and G-4.

The validator accepted `SWAP_EXACT_IN` without decoding its path, so it never saw the pool, the hook or the hook data. Uniswap v4 runs the hook's `beforeSwap` during the swap, and a hook can call any token with any approval the wallet has left behind. The fork test used the deployed Universal Router and PoolManager on Arbitrum, with the hook placed at a test address.

Fix. Each hop is decoded; the input currency and every intermediate currency are declared for measurement; native legs are refused; and each hop's hook must be none or approved in the hook registry. An extra fork test confirms the validator reads the path exactly as the router does, including Universal Router 2.1.2, which adds a field to the v4 swap.

BVCC-24 HIGH Fixed before deployment

An agent withdrawal ran a frozen hook

LOCATION	`BVCCPositionManagerValidatorV5.sol` — `DECREASE_LIQUIDITY` / `BURN_POSITION`
IMPACT	A hook frozen by governance still ran when an agent withdrew liquidity, and could spend old owner approvals: 1e18 units of an unlisted token in the fork test.
LIKELIHOOD	Low: needs a hook that was approved, then frozen as malicious, and an old owner approval to it.
SOURCE	V5 round 4, `R4-H01`. Breaks G-2 and G-3.

Removing liquidity was deliberately left unchecked so an owner could always leave a pool whose hook had been frozen. But Uniswap v4 calls `beforeRemoveLiquidity` on removals too, so a frozen hook kept running whenever an agent withdrew.

Fix. For agents, `DECREASE` and `BURN` resolve the position's pool and require its hook to be none or approved. Leaving a pool with a frozen hook is now the owner's call, through the owner's own execution, which this validator does not govern. Adding, removing and burning also declare both pool currencies for measurement. The trade-off is in §6: "you can always leave" is not guaranteed when a hook is frozen, because the hook may still act or make the exit fail.

BVCC-25 HIGH Accepted — trust decision

An approved v4 hook runs code the wallet cannot bound

LOCATION	`BVCCUniversalRouterValidatorV5.sol`, `BVCCPositionManagerValidatorV5.sol` — hook checks
IMPACT	An approved hook that is malicious or compromised can do what the unapproved hook of BVCC-23 did: spend old owner approvals of tokens the agent may not use.
LIKELIHOOD	Low: needs governance to approve a hook that is, or later becomes, hostile. Approval takes 48 public hours and a freeze is immediate.
SOURCE	V5 round 4, `R4-H02`; already recorded as a trust decision in the specification.

The validators check a hook's *address* against the hook registry. They cannot limit what its code does, what hook data it receives, or which approvals it uses. Pinning the code hash would detect a bytecode change but would not prove the code harmless, nor catch state changes or a proxy upgrade.

Decision. Hooks stay available to agents, so G-2, G-3 and G-4 hold on the condition that approved hooks are trustworthy. Each hook is reviewed before it is proposed, upgradeable hooks (proxies) are never approved, every approval waits 48 hours in public, and a hook can be frozen at once. Agents that only use pools without hooks are not exposed. Owners reduce the exposure further by not leaving old token approvals to hook contracts.

BVCC-26 HIGH Fixed before deployment

An agent could drain a third party's position that approved the wallet

LOCATION	`BVCCPositionManagerValidatorV5.sol` — `DECREASE_LIQUIDITY` / `BURN_POSITION`
IMPACT	Total loss of a *third party's* v4 position, NFT included with the burn variant. The wallet stayed within its limits: the withdrawn tokens were charged to the agent's budget, then sent to an allowed recipient.
LIKELIHOOD	Low: needs a third party who approved the wallet as operator on their position.
SOURCE	Red team, `RT-01`. Reproduced on forks of five networks, against five deployed PositionManager versions.

This is BVCC-21 in reverse. The owner check had been added to `INCREASE_LIQUIDITY` only, and the PositionManager accepts approved operators for every action, so an agent could withdraw and burn a position the wallet did not own.

Fix. `DECREASE` and `BURN` now require `ownerOf(tokenId) == wallet` as well. Five network forks reject the foreign position for both actions. Round 5 asked for tests of removing and burning the same position in one call, in both orders, for own and foreign positions; they were added, including one on real Arbitrum state.

BVCC-27 MEDIUM Fixed before deployment

Agents could never burn a v4 position

LOCATION	`BVCCAgentLogicV5.sol` — calldata scan
IMPACT	Denial of service, no theft: an agent could open v4 positions but every burn reverted, so it could never close one.
LIKELIHOOD	Certain for any agent managing v4 liquidity.
SOURCE	Red team, `RT-55`.

The scanner treats any contract that answers `balanceOf` as a token. The PositionManager is an ERC-721 and answers with the number of positions the wallet holds, so burning one looked like an outflow of a token the agent was not allowed to use, and the batch reverted with `TokenNotAllowed`. The owner's identical burn worked.

Fix. The scanner skips contracts that declare ERC-721 through ERC-165, with a capped, read-only probe. Tokens on the agent's allowed list and tokens declared by validators are still measured first. Round 5 checked that a fake "ERC-721" cannot use this to hide a token that is allowed or declared.

BVCC-28 LOW Fixed before deployment

`pauseAgents` reverted when the wallet was already paused

LOCATION	`BVCCAgentWalletV5.sol` — `pauseAgents` / `unpauseAgents`
IMPACT	An owner batch that paused all agents failed if they were already paused, for instance after a recovery. No funds at risk.
LIKELIHOOD	Ordinary owner use.
SOURCE	Red team, `RT-30-A`.

Fix. Both functions now do nothing when the wallet is already in the requested state. The events are still emitted, so anything watching the wallet should read `paused()` for the state.

BVCC-29 MEDIUM Accepted — trust assumption

An allowed token that lies about its balance or moves another token

LOCATION	Token contracts on the agent's allowed list
IMPACT	A malicious allowed token can leave without a charge by reporting a false balance, or can sweep a different token out of the wallet when it is transferred.
LIKELIHOOD	Low: the owner must put such a token on the agent's list.
SOURCE	Red team, `RT-52` (Low) and `RT-54` (Medium), combined here.

V5 measures balances, and a balance is whatever the token contract says it is. A token can also run arbitrary code in `transfer`, including a transfer of a second token it has permission to move. No wallet can defend against a token that lies about itself.

Decision. Allowed tokens are trusted, in the same way as an approved hook: choosing them is the owner's decision. The agent list should hold standard ERC-20s. The same-token tax case fails closed (BVCC-18).

BVCC-30 LOW Accepted — validator rule

The wallet only measures tokens it can see or a validator declares

LOCATION	`BVCCAgentLogicV5.sol` — calldata scan; every validator
IMPACT	A token packed inside calldata and not declared by the call's validator is not measured. With a validator that under-declares, it could leave unbudgeted.
LIKELIHOOD	Low with the deployed validators, which each declare the tokens of every action they accept. Becomes Medium if a registered validator under-declares.
SOURCE	Red team, `RT-65`.

This is the residual of BVCC-15. The scanner reads whole 32-byte words; a packed address is invisible to it, and so is a token whose balance probe needs more than the gas cap and is not on the allowed list.

Rule. A validator must declare every token its target can move and refuse whatever it cannot decode. The Universal Router validator declares every currency of the path, the position-manager validators declare both pool currencies, Aave declares the asset, and the wrapped-native validator declares the wrapped token.

5 Remediation & Deployment Verification

V5 is the generation that is live today, on six networks. This section records what is deployed, how it was checked independently of the deployment scripts, and what has been exercised on real networks. The V4 remediation record follows as the previous generation.

V5 deployment — Ethereum · Arbitrum One · Base · BNB Chain · Polygon · Arbitrum Sepolia

CONTRACT	ADDRESS (CREATE2, IDENTICAL ON EVERY NETWORK)
Agent logic library (`BVCCAgentLogicV5`)	`0x7504bf3c1230eF99eaEFAc03287c8F036CBb6D0E`
ValidatorRegistry V5	`0x7269334AE74DCE4e607eec36268383DD35f28350`
HookRegistry V5	`0xEaEaC7F6071f7F576Be2aD97898Dd0c0F51BE78A`
SmartWallet logic V5	`0xc9dE89d54beB786bc916131CA6c8c5374507Bd15`
AgentWallet logic V5	`0x360f991B23a3D5202Dc422292697aB07dA279cFA`
SmartWalletFactory V5	`0x2672951DC1E3f865BA6Fc98e5509dc4B23B260a2`
AgentWalletFactory V5	`0xE36fb66c21C38536454C1a58fBC5EB1Cb455aD52`

Everything was deployed through the standard CREATE2 deployer (`0x4e59…956C`): Arbitrum Sepolia on 2026-10-30, then the five mainnets on 2026-11-02, from the code frozen on 2026-10-29. The deploy script refuses a mainnet run unless the two factories land on the pinned addresses above. The wallets are ERC-1167 clones of the two logic contracts; they cannot be upgraded. The registries and the factories' kill switch belong to the governance hardware wallet `0x3145Bd5e…5A4c`, with ownership transferable only in two steps, and fees go to `0x3e3eb089…07D4`, a constant compiled into the logic. Neither address can move user funds: the registries hold nothing, and `kill` only stops new wallets.

Creating a wallet costs about 537,000 gas, against about 4.7 million for V4, because each wallet is a small clone. The agent wallet logic is 22,604 bytes, 1,972 below the EIP-170 limit, with the permission code moved into the external library.

V5 validators by network

Validators are bound to one protocol contract each, so most addresses differ per network. All 30 were read back from `validators(target)` on the V5 registry on 2026-11-11: each is **active** and points at the expected validator. The first three on Arbitrum Sepolia were activated on 2026-11-01 and the other 27 between 2026-11-04 and 2026-11-08. Each one went through the registry's 48-hour public delay.

NETWORK	VALIDATOR FOR	ADDRESS
Ethereum	Universal Router	`0x48A6d5415C369Fa38E0ebEafC779527fb6cF32A0`
Ethereum	Uniswap v4 PositionManager	`0xb05fc3aaD6E4cAaD69b680C1adae645DB0B56f1f`
Ethereum	Aave v3 Pool	`0xeD51c5063dB3bB4609Cf95fC349d87b68bEECC28`
Ethereum	Uniswap v3 positions	`0xfcf4AB3E1FC5b32A00161e66ebe537B617fC9678`
Ethereum	Wrapped native	`0xDC2480bD3DfDC5A4c92ab1C37f2234FF5f91521F`

Arbitrum One	Universal Router	`0x332E88FDc5f26a7421534902Cbb9C95B70E8AA0d`
Arbitrum One	Uniswap v4 PositionManager	`0xc9Ae38709cd76b51F4f11a21Ee2aAC1cf67a013D`
Arbitrum One	Aave v3 Pool	`0xC7bD86E147b211DDC1c6bbCad07A01B51C7CAff4`
Arbitrum One	Uniswap v3 positions	`0xfcf4AB3E1FC5b32A00161e66ebe537B617fC9678`
Arbitrum One	Wrapped native	`0x80DBf05b4406927c22cDFaFb3f226665F863A58D`
Base	Universal Router	`0x5b9cDCC1a4Ac139E8b13A4FC4ca4519906F651A5`
Base	Uniswap v4 PositionManager	`0x0E43866CFFC41E814C1f4e0341Fed09234d1d8B1`
Base	Aave v3 Pool	`0x30Bd59CF41c2F4FE02e9666065dd294FD44333bb`
Base	Uniswap v3 positions	`0x2B52943E9F361CCA42f1100a78b74daaEB25F02e`
Base	Wrapped native	`0xEF913B99C970E4Bc3B987F0668d0F20CD8B90c6E`
BNB Chain	Universal Router	`0x656f0EB67ce76c3b416D8F3dFB46e81c7aA442D7`
BNB Chain	Uniswap v4 PositionManager	`0x12298d84D1d3272a51a7b21755D222da26996786`
BNB Chain	Aave v3 Pool	`0x822E3ab17d7AC4E49cA18E881392CED15ba1E7bb`
BNB Chain	Uniswap v3 positions	`0x5C700655e00FA7003672617d8723d5AeBF9E9c9A`
BNB Chain	Wrapped native	`0x4F2974859169262158065a1506070ee52EFeD6De`
Polygon	Universal Router	`0xDd313C3884a6fA1636fc5cba3E251874073e1B3f`
Polygon	Uniswap v4 PositionManager	`0xd5315B0B30Fa25C5EcbFAA2BBD514262E33Db911`
Polygon	Aave v3 Pool	`0xC7bD86E147b211DDC1c6bbCad07A01B51C7CAff4`
Polygon	Uniswap v3 positions	`0xfcf4AB3E1FC5b32A00161e66ebe537B617fC9678`
Polygon	Wrapped native	`0xc4E512BEEbBCb03A79C10c109B4CD6cef8aa5E5e`
Arbitrum Sepolia	Universal Router	`0x1D0394e997a0799883AE433383Bc82280E17E403`
Arbitrum Sepolia	Uniswap v4 PositionManager	`0x6Cd6e2D7328b0128Ab82921939526dde6f45F2d3`
Arbitrum Sepolia	Aave v3 Pool	`0xA61ac81B9fAA48eE33Ac8A1f28ad80C577e276FA`
Arbitrum Sepolia	Uniswap v3 positions	`0xbaBA421723a25363BB25468B94A1100B3972DB84`
Arbitrum Sepolia	Wrapped native	`0x0A9F4027e3A2136d4e688D0Ee0A5b42cE94b3702`

> **72 of 72 contracts source-verified.** Twelve contracts per network (the seven core contracts and five validators), checked from outside through the block explorers' API on 2026-11-11: every one is verified, on all six networks. Anyone can compare the published source with the tree named in §2.

V5 on real networks

WHERE	WHAT WAS RUN	RESULT
Arbitrum Sepolia, 2026-11-01 → 11-08	An agent driven through the MCP server against a V5 agent wallet: swaps on Uniswap v3 and v4 and from native, Aave supply, borrow, repay and withdraw, Uniswap v4 and v3 positions opened, collected, reduced, closed	All as designed. Over-limit, foreign-recipient and unsupported operations were refused with the expected error before or at execution

	and burned, wrap and unwrap, sends, and each limit tried on purpose	
Arbitrum Sepolia, 2026-11-06	Guardian recovery: two guardians voted the same new passkey, the proposal armed, 48 hours later it was executed	Owner key and credential rotated; every agent paused, as specified; the owner resumed them with the new passkey
BNB Chain, 2026-11-11	Migration of a V4 agent wallet to a V5 agent wallet (`0x566669C6…4936`) with "Send everything": one owner signature, one batch	`0x549492c6…91724b` (BNB) and `0xf0141064…ac27a16` (USDC and BNB together): amounts and the 0.15% fee exact

> Before deployment, the production deployment was also simulated on forks of all six networks with the real fee and admin wallets, including an owner operation through the real EntryPoint (v0.9) and an agent authorized by UserOp and operating. **What has not been run yet** is an agent operating a V5 wallet on a mainnet. The agent path is covered on Arbitrum Sepolia and on mainnet forks; the claim this report makes for mainnet is "deployed, verified and wired", not "exercised by an agent".

V4 remediation (previous generation)

Nine findings of the V4 review are remediated in V4. Seven required a bytecode change and shipped together as the V4 generation; the other two were closed before the affected code reached a network. The rest of this section records what is live for V4, where, and how that was established.

Seven findings required a bytecode change. All seven are fixed and **live on all six networks**. Because CREATE2 addresses derive from the bytecode, they could not be applied to wallets already deployed: the migration is the same playbook as V1→V2 and V2→V3 — users recreate their wallet and move funds, after which the superseded factories are killed. They travelled in one deployment rather than forcing several migrations. The contracts were renamed to V4 and the EIP-712 domain follows (`BVCCSmartWalletV4`), as in the V2→V3 generation; the interface reads that domain to detect an outdated wallet and offer the migration.

ITEM	STATUS
Cross-function reentrancy (BVCC-01)	**Fixed** two layers, each with its own regression test
Guardian squatting (BVCC-04)	**Fixed** the factory no longer chooses guardians; `setGuardians` is passkey-gated
Unauthenticated `credentialId` (BVCC-06)	**Fixed** moved out of the factory into a wallet-emitted, signed event
Credential stale after recovery (BVCC-07)	**Fixed** `setCredentialId`, self-call gated
`approve` carrying ETH (BVCC-11)	**Fixed** token calls must carry zero value
Agents survive recovery (BVCC-05)	**Fixed** recovery pauses agents
Guardians could not be rotated (BVCC-08)	**Fixed** owner may replace the set, never during a recovery
Case-3 token accounting (BVCC-03)	**Open** staged; the validator stage is what closes it
Single-guardian recovery stall (BVCC-09)	**Open** accepted; denial of service, not theft

Zero-check in validator constructors (BVCC-13)	**Open** accepted; inert — every deployed instance was read back and both immutables are non-zero
Allowance revocation UX, token-cap defaults (BVCC-10)	**Open** interface work — deliberately not put in the contract

V4 deployment — Arbitrum One · Ethereum · BNB Chain · Base · Polygon · Arbitrum Sepolia

CONTRACT	ADDRESS (CREATE2, IDENTICAL ON EVERY NETWORK)
SmartWalletFactory V4 (salt …0A)	`0xfd105197109244483b5f870501326E6faec9F93c`
AgentWalletFactory V4 (salt …0B)	`0xf3A61F9d64d45362E149A111289546523BCd26a6`
ValidatorRegistry	`0x5e371D54AC97a57B0a99145Ed04A3c9fA07850C2 — unchanged`

Verified independently of the deployment script by reading each network back: the deployed bytecode is identical across all six (smart wallet 15,959 bytes, agent wallet 24,541), the owner is the governance hardware wallet `0x3145Bd5e…A4c`, and the registry is untouched. The factories are verified on the block explorers.

The registry is deliberately untouched: it is the address compiled into every wallet as a constant, and the freeze-guard test proves the V4 wallet still resolves to it. Validators are shared across generations, so nothing had to be re-registered for V4.

Liquidity on v4 — exercised end-to-end by an agent on mainnet

The deep-validation path that V3 introduced and V4 inherits was run in full by an agent on Arbitrum One, against a native pool — the case bitmap pinning cannot express, and the reason `BVCCPositionManagerValidator` exists. Pool ETH/USDC, fee 500, tick spacing 10, no hooks, full range.

STEP	TRANSACTION	GAS	RESULT
mint	`0xb4245927…f1269`	622,967	tokenId 190877, `ownerOf` = wallet
read position	—	—	currency0 native, currency1 USDC, liquidity 4,377,897,532
collect	`0x9296f6f3…a865c7`	218,749	success, zero proceeds, zero fee
decrease 100% + burn	`0x92cce43d…dca9ef`	267,739	NFT burned

The validator behaved as designed throughout: the `TAKE_PAIR` and `SWEEP` actions were accepted with `recipient` resolving to the wallet, and the dry-run passed as soon as the PositionManager was added to `allowedProtocols`. The mint sent 0.0001 ETH and the `SWEEP` returned the 17,050 wei of native surplus, so the PositionManager was left holding **zero ETH** — the stranded-value case the validator refuses when no native `SETTLE` consumes the value. Final state re-read on-chain for this report: `ownerOf(190877)` reverts `NOT_MINTED`, position liquidity is 0, PositionManager native balance is 0.

> This cycle was exercised on **Arbitrum One only**. On the other five networks the validators are deployed, registered and confirmed bound to the correct PositionManager, but the end-to-end liquidity cycle has not been run. The claim this report makes is therefore: proven end-to-end on Arbitrum, deployed and verified identical elsewhere.

The agent factory ends at **24,541 bytes**, 35 below the EIP-170 limit, with `executeAsAgent` case 3 at 64,972 gas. That margin is thin, and it is the reason two candidate hardening measures were deliberately kept out of the contract: requiring non-zero token caps when an agent has protocol access, and capping the owner's own approvals while an agent is active. Both belong in the interface, where they cost nothing and can explain themselves. The remaining bytecode is reserved for measures with no off-chain equivalent.

6 Residual Risk & Recommendations

Nothing is open in V5. Four findings are accepted as trust assumptions, V5 has known limits that an owner should understand, and wallets still on V4 or earlier keep their own open findings until their owners migrate. This section says what an owner should do.

First: migrate

A deployed wallet cannot be upgraded in place. A V4 wallet still has **BVCC-03** open: where the owner has granted a token allowance to a protocol an agent can reach, a compromised agent key is worth more than its budget. Wallets on V1 to V3 also run the Critical BVCC-01. The fix is to create a V5 wallet with the same passkey and move the funds. The app detects an outdated wallet, lists what V5 changes, and provides a migration guide and a "Send everything" button that moves every selected token on a network in one signature. Positions in Uniswap or Aave belong to the old address and must be closed first; agents must be authorized again on the new wallet and revoked on the old one.

What V5 asks the owner to trust

- **Approved Uniswap v4 hooks (BVCC-25).** An approved hook runs its own code during agent swaps and liquidity operations and can reach old owner approvals. Each hook approval waits 48 hours in public and can be frozen at once; agents that stick to pools without hooks are not exposed.
- **Allowed tokens (BVCC-29).** A token that lies about its balance, or moves another token when transferred, defeats any balance-based limit. Give agents standard ERC-20s only.
- **Validators (BVCC-19, BVCC-30).** Only immutable, stateless validators are registered, and each must declare every token it lets move. The five deployed validators meet both rules, and their source is verified on every network.
- **The owner's own approvals.** Allowances the owner grants from any dApp live in the token contracts. An agent cannot use them beyond its budget, but a hostile approved hook could (BVCC-25), so revoke the ones you no longer need.

Known limits of V5

- **Not upgradeable.** A serious bug would need a V6 and another migration.
- **Agents must be plain EOAs**, checked at authorization and at every call. An agent address with an EIP-7702 delegation stops working.
- **Limits mean what they say.** 0 allows nothing, an empty recipient list allows no one, and "unlimited" must be chosen explicitly. Agent permissions expire within 365 days and can be renewed.
- **The fee is charged on top**, at 0.15% of what leaves the wallet, inside the agent's limits. An agent cannot send its whole balance of a token: it must leave room for the fee.
- **Tokens that tax the sender** cannot be moved by agents (BVCC-18). Tokens that tax the receiver work.
- **With a frozen hook**, only the owner can try to leave that pool, and the hook may still act or make the exit fail (BVCC-24).
- **Moving liquidity between positions** takes two calls, and what comes out pays 0.15% as an inflow (BVCC-22). The native-ETH part of leaving a v4 position is not charged on the way in.
- **Recovery is per network.** After losing the original passkey, a wallet cannot be recovered on a network where it was never deployed: it would be born with the lost key and no guardians. Deploy on every network you may use.

- **Agents keep operating while a recovery is armed** and are paused when it executes. This was a deliberate choice: a recovery means a lost passkey, not a stolen one, and pausing for 48 hours could stop an agent from managing an Aave loan.
- **NFTs are not measured.** Agents reach positions only through validators, and what comes out of a position into the wallet is charged.

Operational recommendations

1. **Migrate** every V4 or older wallet that holds funds, on every network.
2. **Deploy on every supported network** so the address is yours everywhere and recovery works there.
3. **Give each agent the least it needs**: specific tokens, limits, recipients and an expiry. Pause a single agent rather than all of them when only one misbehaves.
4. **Revoke token approvals you no longer use**, especially approvals to hook contracts.
5. **Watch the registries.** Any widening of what agents can do is announced on-chain 48 hours before it takes effect; pausing or revoking agents in that window is immediate.

Wallets still on V4 or earlier

Until they migrate, these owners keep the V4 picture. Anchoring a destination is not the same as bounding a value: where a standing allowance exists, a compromised agent key can swap a valuable asset for a worthless one in a pool the attacker controls, and the value stays with the attacker. The interim measure needs no contract change: **zero any standing allowance towards the protocols an agent can reach before authorizing it**, and prefer per-transaction approvals. BVCC-09 (a single guardian can stall a recovery) and BVCC-10 (pausing an agent does not retract its allowances) also still apply there. Funds left on V1, V2 or V3 run bytecode with BVCC-01 unfixed; on those wallets a non-empty recipient whitelist that excludes the agent's own address is what closes it.

7 Conclusion & Limitations

V5. The V5 generation replaces "check where the money may go" with "measure what actually left". That change closes BVCC-03, the one material risk V4 left open, and with it the class of problems where a standing allowance or a cleverly combined batch let an agent spend outside its budget. V5 also keeps every V4 fix, closes the recovery stall of BVCC-09, removes the agent's ability to leave allowances behind, makes 0 mean nothing and every permission expire, and requires a validator on every protocol call. Five review rounds and a red-team exercise found seventeen issues in the new code before it shipped, thirteen fixed and four accepted as stated trust assumptions. The last round found nothing new. V5 is deployed on six networks with all 72 contracts source-verified and all 30 validators active.

Most of the V5 findings came from one place: validators for Uniswap positions, where value can move between NFTs without any token leaving the wallet. Each fix there was followed by a finding next to it, until the rules became symmetric (owner checks on every action, no remove-and-add in one call, hooks checked on every path). That pattern is the main reason to keep reviewing V5, and Appendix G records it.

Earlier generations. The BVCC Agent Wallet enforces a comprehensive, layered permission model. Three high-severity issues were identified across three review rounds, each reproduced, remediated, redeployed and re-verified, and the on-chain permission battery passes on the patched bytecode. Four more surfaced while preparing this edition — a cross-function reentrancy, guardian squatting through the factory, the unbounded case-3 spending of BVCC-03, and agents surviving a recovery. All but one are fixed and deployed in the V4 factory generation, together with the credential, guardian-rotation and `approve`-with-value fixes. Guardian recovery was executed and verified end-to-end — ownership rotation to a new passkey plus on-chain rejection of the old key (see Appendix D). The V2 round fixed a mainnet gas-griefing issue in the fee snapshot and concluded with a deterministic multichain deployment (see Appendix D).

The V3 round is the deepest change to the agent path so far. It closed a fund-exfiltration vector that budgets could not see, by making agent DeFi calls default-deny per selector and anchoring the destination either to the wallet itself or to an on-chain validator whose registration is asymmetric — immediate to deny, 48 hours to allow (see Appendix A, Appendix A). Two would-be bypasses were caught inside that work: a command mask that would have accepted a cross-chain bridge deposit as a swap, and an `increaseLiquidity` path with no owner check, which was deliberately left unregistered. The model is verified by 264 unit, fork and fuzz tests, by a mainnet swap in which every recipient resolves to the wallet and the fee is exact, and by the adversarial variant of that same swap being rejected before execution (see Appendix B).

V4 closed that round and is live on all six networks, with the deep-validation layer complete: both selectors that defer to the registry — the Universal Router's `execute` and the v4 PositionManager's `modifyLiquidities` — have an active validator everywhere, and the full liquidity cycle was exercised on Arbitrum mainnet by an agent, ending with the position burned and no native value stranded in the PositionManager (see Appendix A, Appendix D).

The residual risk is stated plainly rather than minimised. Case-3 protocol calls are safe in the sense V3 set out to make them — the destination is anchored — but anchoring the destination is not the same as bounding the value, and where a standing allowance exists that difference is the whole loss. That gap survives V4: a compromised agent key on a wallet left in the permissive default, with an allowance already granted, is still worth more than its budget. The staged plan in BVCC-03 says which part the interface can narrow now and which needs the validator layer to close properly. Separately, every finding in this report still applies to funds sitting in V1, V2 or V3 wallets, because a

deployed wallet cannot be upgraded in place — the exposure ends when the user migrates, not when V4 ships.

What this review does and does not establish

It establishes that the specific findings recorded here were reproduced, remediated where stated, and that the remediation is present in bytecode deployed on the networks listed in Appendix F — each read back from chain rather than inferred from a deployment script. Every finding has a regression test, and the suite passes. For V5, it also establishes that the deployed contracts are source-verified (72 of 72) and that the 30 validators are active and point where they should.

It does **not** establish that the contracts are free of other defects. This is an internal review by the team that wrote the code. The four findings contributed by a later pass with different tooling — BVCC-04, BVCC-06, BVCC-07 and BVCC-11 — had each survived three earlier rounds, which is the clearest available measure of what repeating the same method misses. **No independent party has reviewed this code**, so nothing here has been checked by someone with an interest in finding fault. The contracts have not been externally audited, and this document should not be read as a substitute for one.

The V5 rounds were run with AI analysis tools, directed by the same team. They found more before deployment than any earlier round did, and they also showed their limit: rounds 2 to 4 each found a path the previous round had not looked for. The fifth round's auditor could not compare the code with round 4's on its own; the team made that comparison. An AI review is a strong second pair of eyes, but it is not an independent audit.

Coverage is also uneven by network. For V4, the permission battery ran on Arbitrum Sepolia; the call-policy layer and the Uniswap v4 liquidity cycle were exercised on Arbitrum One. On the other four networks the contracts are deployed and verified byte-identical, and the validator wiring was read back, but no end-to-end cycle was run. For V5, the full agent battery ran on Arbitrum Sepolia and on mainnet forks; on mainnet, owner operations have run (BNB Chain) but no agent has yet operated a V5 wallet.

A Security Architecture

Reference material for the mechanisms the findings refer to. The V5 design comes first because it is the current generation; the V3 and V4 call-policy layer and validator registry follow, since V5 builds on them and V4 wallets still use them.

V5 — what it guarantees

The V5 specification states seven guarantees. Each is an invariant the test suite checks, several of them with 128,000 random action sequences.

ID	GUARANTEE
`G-1`	Only the owner's passkey, or a 2-of-3 guardian recovery completed after 48 hours, can change who owns the wallet.
`G-2`	An agent never takes more of a token out of the wallet than its budget for that token, fee included, whatever approvals exist and however the calls are combined.
`G-3`	An agent never moves a token it is not allowed to use.
`G-4`	An agent never sends funds to a destination it is not allowed to use, directly or through a protocol.
`G-5`	An agent cannot call the wallet's own functions or regain owner control mid-operation (reentrancy).
`G-6`	External governance can only restrict at once; widening takes 48 hours visible on-chain. Requires immutable, stateless validators (BVCC-19).
`G-7`	No one but the owner sets the wallet's credential or guardians, including whoever deploys it.

G-2, G-3 and G-4 hold on the condition that approved hooks and allowed tokens are trustworthy (BVCC-25, BVCC-29).

V5 — the agent batch, step by step

The agent sends its own transaction to `executeAsAgent(mode, executionData)`; only the ERC-7579 batch mode is accepted. Inside it:

1. **Agent check.** A plain EOA at this very call, active, not expired, not paused globally and not paused on its own.
2. **Per item, before anything runs.** The target is not the wallet and not blocked by the owner. A native send must go to an allowed recipient; a `transfer` must be an allowed token to an allowed recipient; an `approve` must be an allowed token to an *allowed protocol*. Any other call needs an allowed protocol, a policy for that agent and function with the `DEEP` bit, any anchors the owner set, and a yes from the protocol's validator, which also returns the tokens to measure, any costs (such as a loan), whether it wants a post-check, and whether inflows are charged.
3. **Measure.** Read the balance of every allowed token, every token found in the calldata and every token a validator declared, up to 60. A failed read reverts.
4. **Native.** Charge the native value sent, plus its fee, against the native limits.
5. **Execute** the batch with the reentrancy flag set, measuring charged inflows around the calls that asked for it.
6. **Reset approvals.** Every approval the agent made goes back to its previous value.
7. **Measure again and charge.** For each token that went down: an unallowed token reverts the batch; an allowed one is charged outflow + costs + 0.15% fee against the per-batch, daily and total limits, and the fee is paid in that same token.

8. **Post-checks.** Validators that asked for one run again; for Aave, the health factor must stay at or above 1.2.

V5 — permissions per agent

SETTING	V5	V4
Expiry	Required, at most 365 days ahead; `renewAgent` extends it	0 meant never
Native and token limits (per batch, daily, total, period)	0 = nothing allowed; "unlimited" only as an explicit maximum	0 meant no limit
Allowed tokens	Up to 50, each with its own limits	Up to 20
Recipients	Empty list = nobody	Empty list = anybody
Call policies	Per agent; a validator is required on every protocol call; extra anchors for tokens, recipients and value sets	Per wallet; bitmap pins, validator only for two selectors
Pause	Per agent, plus a global pause	Global only
Blocking a protocol	The owner can block a target for every agent at once	—

V5 — the owner side

- **Creation needs the passkey.** `createWallet` verifies a passkey signature over the wallet address and credential, so nobody can deploy someone else's wallet or choose its guardians (the class of BVCC-04 and BVCC-06, closed for good). The signature carries no chain id on purpose, so the same wallet can be deployed on every network.
- **The credential lives in the wallet** and changes only at creation, by the owner, or by a recovery, which sets the new one (BVCC-07).
- **Owner signatures expire.** Every UserOperation carries a `validUntil` the passkey signs and the EntryPoint enforces; the app uses about 15 minutes.
- **Recovery by votes** on a specific proposal, as described under BVCC-09. Executing it pauses every agent.
- **Owner execution** keeps the three V4 fee cases (native, ERC-20 transfer, other calls) at 0.05%, or 0.15% on an agent wallet, with SafeERC20 semantics and balance reads capped at 32 bytes of return data.
- **Reentrancy** keeps both V4 layers (the EOA check at every call and no external `execute()` during an agent batch), now in transient storage.

V5 — validators

The interface returns five values: yes or no, the tokens to measure, the costs, whether a post-check is wanted, and whether inflows are charged. Validators are `view` contracts called with `staticcall`; a revert means no.

VALIDATOR	WHAT IT ALLOWS AGENTS TO DO, AND HOW IT LIMITS IT
Universal Router	Token-to-token swaps (v3 and v4). Exact command bytes; every recipient is the wallet; every v4 hop decoded, each hook none or approved; all path currencies declared; native value refused.
Uniswap v4 PositionManager	Mint, add, remove, burn, collect. Positions owned by the wallet only; adding needs no individual approval; no remove-and-add in one call; hooks none or approved on every action; both pool currencies declared; inflows charged.

Aave v3 Pool	Supply, withdraw, borrow (variable), repay, collateral and eMode settings. Beneficiary pinned to the wallet; a loan counts as a cost against the budget; withdrawals charged as inflows; health factor ≥ 1.2 after any risky change.
Uniswap v3 positions	Mint to the wallet; add only on own, unapproved positions; remove, burn and collect only own positions, collect to the wallet exactly. Everything else, and any native value, refused.
Wrapped native	Only `deposit()` with value and `withdraw(amount)` without. The value is already charged on the native limits.

Clones and library: each wallet is an ERC-1167 clone of a logic contract whose initializers are disabled; the agent permission code is an external library reached by DELEGATECALL, with no storage of its own. The registry address is a constant inside every wallet.

V3 and V4 — call policies and the validator registry

Agent execution model

The agent calls `executeAsAgent(bytes32 mode, bytes executionData)` as an ordinary EOA transaction (not a UserOp). `mode` is the ERC-7579 batch selector (`0x0100…00`); `executionData` is an ABI-encoded `Execution[]` of `(target, value, callData)`. Validation order inside the call is **per-tx** → **recipient/protocol** → **period** → **daily** → **total**, with state updated **before** external execution (checks-effects-interactions) and a `nonReentrant` guard.

HIGH (agent fund exfiltration) **FIXED — V3 DEPLOYED 2026-07**

Description

Through V2 an agent's budget was charged whenever value left the wallet along a path the wallet could see: a native transfer, an ERC-20 `transfer`, or an `approve`. A protocol call (case 3) was permitted as soon as its target appeared in `allowedProtocols`. That is not enough, because several DeFi entry points take the destination as an argument:

- Aave's `Pool.withdraw(asset, amount, to)` sends the supplied position straight to `to` — no `approve`, no `transfer`, so the budget never saw it.
- Swap and LP calls (`exactInputSingle`, Universal Router `execute`, `mint`) carry a `recipient` argument the agent chose freely.

A stolen or misbehaving agent key could therefore name its own address as the destination and drain everything the wallet had supplied or approved to a whitelisted protocol, **without consuming a single wei of its budget**. Severity is HIGH: the loss is bounded only by the wallet's DeFi exposure, and the on-chain accounting shows nothing.

Remediation (V3) — per-selector call policies

Case-3 calls are now **default-deny per selector**. Whitelisting the protocol is no longer sufficient: the owner must also register a policy that says *where the money is allowed to go*. Policies are written with `setCallPolicy(target, selector, policy)`, callable only by the wallet itself — in practice the owner's passkey, bundled into the same signature that authorizes the agent. The policy is one `uint256` word:

BITS	FIELD	MEANING
255	`POLICY_ALLOWED`	Selector is permitted. Absent → `SelectorNotAllowed`
254	`POLICY_DEEP`	Route the call to the on-chain validator registry (see Appendix A)
223..192	`PIN_WALLET` (32-bit bitmap)	Calldata word *i* must equal `address(this)`

`191..160`	`PIN_PROTOCOL` (32-bit bitmap)	Calldata word *i* must be an address in `allowedProtocols`

Enforcement inside `executeAsAgent` runs before any external call: (1) the selector is read from the calldata (`< 4` bytes → `CalldataTooShort`); (2) the policy must carry `POLICY_ALLOWED`; (3) every pinned word is compared **full-width**, which also rejects dirty high bits, and a word read past the end of the calldata yields zero and therefore fails — the check is fail-closed by construction; (4) if `POLICY_DEEP` is set, the registry must return `true`. Four custom errors were added: `SelectorNotAllowed`, `PinnedArgMismatch`, `CalldataTooShort` and `PolicyValidationFailed`. One related default also changed: an empty `allowedProtocols` list now reverts `NoProtocolsWhitelisted` instead of meaning "any protocol".

> **The owner is never subject to call policies.** They apply only to agents. A human signing with their passkey keeps using any dApp normally — V3 narrows the delegated path, not the owner's.

Registered policies (Arbitrum One shown; the same shapes are used on the other five networks)

PROTOCOL	SELECTORS	ANCHOR
Uniswap SwapRouter02	`exactInputSingle` · `exactOutputSingle`	PIN_WALLET word 3
Uniswap SwapRouter02	`exactInput` · `exactOutput` (multi-hop)	PIN_WALLET word 2
Aave v3 Pool	`supply` · `withdraw` · `borrow` · `repay`	PIN_WALLET word 2 / 2 / 4 / 3
Aave v3 Pool	`repayWithATokens` · `setUserUseReserveAsCollateral` · `setUserEMode`	Allowed, no pin — no destination argument exists
Permit2	`approve(address,address,uint160,uint48)`	PIN_PROTOCOL word 1 (the spender)
Uniswap Universal Router	`execute(bytes,bytes[],uint256)`	DEEP → validator registry
Uniswap v3 NFPM	`mint` · `collect`	PIN_WALLET word 9 / 1
Uniswap v3 NFPM	`decreaseLiquidity` · `burn`	Allowed, no pin — the NFPM gates these on `tokenId` ownership
WETH	`deposit` · `withdraw`	Allowed, no pin — both credit/pay `msg.sender`

> **Two calldata traps found while writing the presets.**
>
> - **Offset slide.** The multi-hop `exactInput` struct carries a `bytes path`, which makes the tuple dynamic and slides an offset word in front of it: the recipient sits at word 2, not word 3 as in the `*Single` variants. Pinning word 3 there would have pinned `amountIn` and left the destination free. Every offset was re-derived by encoding real calldata.
> - **`increaseLiquidity` is deliberately not whitelisted.** Unlike `decreaseLiquidity`, `collect` and `burn`, the v3 NFPM applies no owner check to it — only a deadline check. An agent could therefore have pushed the wallet's NFPM-approved tokens into an attacker-owned `tokenId`, a

recipient bypass that no bitmap pin can express because word 0 is a `tokenId`, not an address. Agents add liquidity by minting a fresh position with the recipient pinned to the wallet; re-enabling `increaseLiquidity` would require a deep validator that checks `ownerOf(tokenId) == wallet`.

What each layer covers

CASE	OPERATION	GOVERNED BY
1	Native ETH transfer	`allowedRecipients` + native limits
2	`transfer(to, amount)`	`allowedRecipients` + token limits
2b	`approve(spender, amount)`	`allowedRecipients` + token limits
3	DeFi / protocol call	`allowedProtocols` + **call policies**

Scope note. V3 closes fund exfiltration through swaps, LP and protocol withdrawals: case 3 is now safe by construction, whatever the owner configures. It does not turn a compromised agent into a no-op — a direct `transfer` or `approve` (case 2/2b) remains governed by the recipient whitelist and the per-token limits, and those are the owner's choice. Simulated on mainnet against the test agent, with an empty recipient list and limits left at zero, a direct `USDC.transfer` to an attacker passes; with recipients and limits configured it reverts `RecipientNotAllowed`. The on-chain limits are the security model.

Why a second mechanism

A bitmap pin only reaches a word at a fixed offset. In the Universal Router's `execute(bytes commands, bytes[] inputs, uint256 deadline)` — and in Uniswap v4's `modifyLiquidities` — the recipient is buried inside variable-length data, so no fixed offset exists. Those selectors carry the `DEEP` bit instead, which defers the decision to a stateless on-chain validator.

Design

- **Fixed dispatch address.** `BVCCValidatorRegistry` is compiled into the wallet as a constant (`0x5e371D54AC97a57B0a99145Ed04A3c9fA07850C2`). Because the policy word carries only a flag and never an address, an owner cannot be tricked into pointing a policy at an attacker's validator. A dedicated test (`Create2Consistency.t.sol`) fails the build if that constant ever drifts from the registry's CREATE2 address.
- **Fail-closed dispatch.** No validator registered for the target → `false`. A validator that reverts, or that has no code, denies as well. Only an explicit `true` lets the call through; anything else reverts `PolicyValidationFailed`.
- **Stateless validators.** Validators are `view` contracts reached by `staticcall`. They never hold funds, never receive approvals and cannot mutate state — they answer yes or no.
- **Dual consent.** Enabling a complex protocol takes two independent steps: BVCC registers the validator in the registry, and the owner registers the `ALLOWED|DEEP` policy with their passkey. Either one missing means the call is denied.

Asymmetric governance

DIRECTION	FUNCTION	DELAY
Allow — register or replace a validator	`proposeValidator` → `activateValidator`	**48 h timelock** + `ValidatorProposed` event

Deny — cancel a pending proposal	`cancelProposal`	Immediate
Deny — disable an active validator	`freezeValidator`	Immediate

The delay runs only in the permissive direction, so a malicious or mistaken proposal is public for two days before it can take effect and owners can react (pause or revoke their agents). The worst case with a stolen admin key is therefore denial of service, or — after 48 publicly visible hours — a degradation to V2-level protection on one protocol. It is never a direct path to funds: the registry moves nothing.

The Universal Router validator

- **Bound to one router.** The router address is an immutable constructor argument; a call with any other target returns `false`. A new router needs its own validator and its own 48-hour governance cycle.
- **Exact command-byte matching** over `{0x00 V3_SWAP_EXACT_IN, 0x01 V3_SWAP_EXACT_OUT, 0x10 V4_SWAP}`. Every other byte is denied, including allow-revert variants and unknown commands.
- **Every recipient is checked**, including the v4 `TAKE` action (`{0x07 SWAP, 0x0b SETTLE, 0x0e TAKE}`): it must be the wallet or the `MSG_SENDER` sentinel. `SWEEP`, `TRANSFER`, `PAY_PORTION`, `WRAP` and `UNWRAP` are denied outright.
- **Malformed input denies.** Undecodable calldata reverts inside the validator, which the registry treats as a denial; an empty `commands` string is rejected explicitly.
- **Scope.** Token→token swaps. Native legs relying on the router's `ADDRESS_THIS` are deliberately out of scope; native swaps are executed as BVCC's own batches through WETH, so the router never holds wallet funds mid-flight.

> **Critical bug caught during review — the command mask.** The first version of the validator masked command bytes with `& 0x3f`. The real router masks with `& 0x7f` (bit `0x80` is `FLAG_ALLOW_REVERT`), which means `0x40` is **not** an alias of `0x00` — it is `ACROSS_V4_DEPOSIT_V3`, a cross-chain bridge deposit. Under the wrong mask the validator would have accepted it as a v3 swap while the router bridged the wallet's funds to an attacker on another chain: precisely the exfiltration class V3 exists to prevent. The fix replaced masking with exact matching, bound the validator to a single router and rejected empty command strings. The behaviour is now fuzzed across all 256 command bytes against a model of the router, and was re-confirmed on mainnet after deployment (see Appendix B).

Aave needs no validator

Aave's destination arguments (`onBehalfOf`, `to`) sit at fixed offsets, so they are handled by bitmap pins inside the wallet itself. No Aave policy carries the `DEEP` bit, the registry is never consulted for Aave, and no timelock is involved. Only operations whose safety cannot be phrased as "word *N* must be the wallet" — flash loans, multicall, adapter routes — would require a validator, and those remain out of scope.

Registered validators — complete on every network

Exactly two selectors carry the `DEEP` bit and therefore consult the registry: `execute(bytes,bytes[],uint256)` on the Universal Router, and `modifyLiquidities(bytes,uint256)` on the Uniswap v4 PositionManager. Both are registered and active on all six networks, read back from `validators(address)` on the registry:

NETWORK	UNIVERSALROUTER VALIDATOR	POSITIONMANAGER V4 VALIDATOR

Arbitrum One	`0xD1522594e185C882bfDEc6FFD4DE8a53F69AF0Ca`	`0x6aE9c36121D5cD2d4Bde816F7F857AfB36Ccb8Eb`
Ethereum	`0x3615a0Fc0663C0201B543deC9816d5Ef30a82ffb`	`0xeFd47Aeb41e532D6a4ed04553b25827d88a4364d`
BNB Chain	`0x6945f861e0D7FCD566739000f3fDC1D44896D815`	`0x5f5Ee1068c2c54bBE896A08444958a289407Cd35`
Polygon	`0xC1C584E4149eD2EFe20DA0155b1B8257232102fF`	`0x62Eb47697BcF63dC9450B662d259AFf555958e25`
Base	`0x4A2Fc73AAdEfC84513defaa0c444d2150AC6A6D8`	`0xa2F35323DAcBFC2b5909B15607b511c6953e78a5`
Arbitrum Sepolia	`0x87550034627966e32d82E3b8AdB3f19c62E8d86E`	`0x32aA1B3E35dc6fF09EB7CDc8F6b43F3573AD2182`

Each PositionManager validator was checked individually against the v4 PositionManager it is bound to through its `POSITION_MANAGER` immutable, and every one resolves to the correct chain-specific address. All six share the same `HOOK_REGISTRY` (`0x551C6e7ABdA04a110790888e711198f25621b066`, CREATE2, code present on each). A crossed binding would have failed closed and denied all liquidity operations silently, which is why it is verified per network rather than inferred from the deployment script.

> The Uniswap **v3** NonfungiblePositionManager has **no** registered validator on any network, and that is correct rather than a gap. Its policies are bitmap pins — `mint` anchors the recipient at word 9, `collect` at word 1, `decreaseLiquidity` and `burn` are owner-gated by the NFPM itself — so no v3 policy carries the `DEEP` bit and the registry is never consulted. `BVCCPositionManagerValidator` is a v4 contract: it is bound to a single v4 PositionManager and decodes v4 action bytes. Registering it against the v3 NFPM would break v3 liquidity, not secure it. `increaseLiquidity` remains deliberately unregistered for the reason given in Appendix A — the NFPM performs no owner check on it.

B Test Evidence

The test batteries behind the findings and their remediations. Every finding in §4 has a regression test. V5 comes first.

V5 test suite — 191 tests: 145 local, 46 on network forks

SUITE	TESTS	FOCUS
`AgentWalletV5.t.sol`	58	Agent permissions, measurement, fees, approvals reset, policies, pause, expiry
`SmartWalletV5.t.sol`	19	Owner execution, the three fee cases, SafeERC20, creation with credential signature
`ValidatorsV5.t.sol`	18	Universal Router, PositionManager v4 and Aave validators, registry and hook registry
`RecoveryV5.t.sol`	17	Votes, arming, withdrawal, cancellation, execution and agent pause
`AgentInvariantsV5.t.sol` · `RecoveryInvariantsV5.t.sol`	11	Invariants with 128,000 random actions each: limits, allowed tokens, owner exit, agent approvals, counters, atomicity; recovery arming and timing
`SignaturesV5.t.sol`	7	ERC-7739 on clones, signature expiry, cross-wallet and cross-chain replay
`GasV5.t.sol` · `DeployV5.t.sol`	11	Gas bounds; CREATE2 addresses and wiring
`RedTeamV5.t.sol` · `AuditRound2PoCsV5.t.sol`	4	Red-team and round-2 proofs of concept, now regression tests
`fork/AllChainsDeployV5.t.sol`	6	Production deployment on all six networks with the real wallets; owner and agent operations through the real EntryPoint
`fork/ArbitrumForkV5.t.sol` · `EthereumUsdtForkV5.t.sol`	12	Real Universal Router, Permit2, Aave v3 and USDT
`fork/Audit*ForkV5.t.sol` · `RedTeamForkV5.t.sol`	13	The proofs of concept of rounds 2 to 4 and of the red team, on real Uniswap contracts across five networks
`fork/V3PositionManagerForkV5.t.sol` · `WrappedNativeForkV5.t.sol`	15	The two validators added after the freeze

Fork tests simulate locally against real network state; they send no transactions. The suite also ran with the attack tests of every round: each proof of concept failed on the code it was written against and passes as a regression on the frozen code. A further 13 adversarial tests from the review of the two new validators pass as well, including WBNB on BNB Chain and WPOL on Polygon.

AGENT BATCH (V5)	GAS
Native only, or one token	~142,000 – 148,000
Four tokens measured	~163,000

Twenty tokens	~306,000
Worst case, 60 tokens measured	~956,000

The cost depends on the tokens that agent is allowed, not on the maximum. Creating a V5 wallet costs about 537,000 gas. Real-network runs of V5 are in §5.

V1 to V4 batteries

Executed against the patched wallet `0x1ED261…F0C3` with the agent EOA `0x3852…4aB4`. Reverts confirmed by simulation; valid cases as real transactions. Allowed recipients were the two recovery guardians (bot-controlled, funds recoverable).

#	TEST	EXPECTED	RESULT
1	ETH per-tx (0.0002 > max 0.0001)	revert	**ExceedsPerTxLimit**
2	ETH within limit (0.0001)	pass	**Success**
3	USDC per-tx (2 > max 1)	revert	**ExceedsTokenMaxAmount**
4	USDC daily (1+1+1, cap 2) — real txs	3rd reverts	**TokenDailyLimitExceeded**
5	Token send to non-allow-listed dest	revert	**RecipientNotAllowed**
6	ETH send to non-allow-listed dest	revert	**RecipientNotAllowed**
7	Total budget (0.0002+0.0001 = 0.0003) — real txs	next reverts	**AgentBudgetExceeded**
8	Period budget (0.0003+0.0002 = 0.0005) — real txs	next reverts	**PeriodBudgetExceeded**
9	Period auto-rollover (periodStart 0)	window resets on 1st spend	**Verified**
10	Paused (owner pauses)	agent blocked	**EnforcedPause**
11	Expiry (lapsed naturally)	agent blocked	**AgentPermissionsExpired**

> **Observation.** The recipient allow-list applies to **token transfers** as well as ETH — a token send to a non-allow-listed address reverts `RecipientNotAllowed`. Re-authorization via “Edit” does not reset the daily counter (keyed by UTC day); with `periodStart = 0` the first spend rolls the period window.

Privilege-escalation resistance (from the 06-06 session)

An agent targeting the wallet itself to call owner functions (`pauseAgents`, `revokeAgent`, `authorizeAgent`) reverts `AgentCannotCallWallet`; arbitrary contract calls outside the protocol allow-list are blocked. The agent can move only ETH and allow-listed tokens to allow-listed recipients.

Test battery — `forge test`: 264 passed, 0 failed

SUITE	TESTS	FOCUS
`BVCCAgentWallet.t.sol`	50	Agent permission core: limits, budgets, expiry, pause, batches
`UniversalRouterValidator.t.sol`	42	Router corpus, attacker recipients, fuzz over all 256 command bytes

`BVCCAgentWalletAdvanced.t.sol`	40	Fee accounting, `approve` regression tests, edge cases
`CallPolicyAdversarial.t.sol`	34	Policy bypass attempts, registry governance, malformed calldata
`BVCCPositionManagerValidator.t.sol`	26	Uniswap v4 LP calldata validation
`BVCCWallet.t.sol`	23	Base smart account: execution, fees, guardian recovery
`BVCCWalletFactory.t.sol` · `BVCCAgentWalletFactory.t.sol`	18	CREATE2 factories, deterministic addresses, kill switch
`BVCCHookRegistry.t.sol`	11	Hook approval registry and its timelock
`AaveIntegration.t.sol`	10	Forked-mainnet Aave cycle and adversarial variants
`BVCCPMValidatorSdkVectors.t.sol` · `ERC7739*.t.sol` · `Create2Consistency.t.sol` · `GasBench.t.sol`	10	SDK golden vectors, ERC-7739 signatures, address freeze guard, gas

Adversarial policy tests (`CallPolicyAdversarial.t.sol`): Aave `withdraw`/`supply`/`borrow`/`repay` with an external `to` or `onBehalfOf` revert `PinnedArgMismatch`; an unregistered selector reverts `SelectorNotAllowed`; `PIN_PROTOCOL` rejects a spender that is not whitelisted; a DEEP call denies when the validator returns false, reverts, has no code or is not registered, and passes only on `true`; the 48-hour timelock, freeze and cancel paths behave as specified; `policy = 0` revokes; short calldata, out-of-range pins and dirty high bits are all rejected; and a batch containing one bad item reverts as a whole.

Aave on a mainnet fork (`AaveIntegration.t.sol`): a full supply → borrow → repay → withdraw cycle, with the 0.15% agent fee charged exactly on `withdraw` and `borrow` and not at all on `supply` and `repay`; budget consumption verified through the `approve` path; an external beneficiary on any of the four calls reverts; `flashLoanSimple` and an unregistered `setUserEMode` revert; a Pool missing from `allowedRecipients` reverts; and the approve+supply batch rolls back atomically on failure.

The permission layer had been attacked from every angle across three rounds; guardian recovery and the signature path had not. Both hold funds, so both were given the same treatment: twenty tests written to make something work that must not (`test/RecoveryAdversarial.t.sol`, `test/SignatureAdversarial.t.sol`).

14.1 Signature path — no findings

Ten attacks against the ERC-7739 / WebAuthn path, all rejected. The two that matter most are the replay pairs, because this wallet's design invites both: the address is identical on every network, and one passkey backs a smart wallet and an agent wallet at two different addresses.

ATTACK	RESULT
Replay a signature from one chain onto the same address on another	**Rejected** chainId is bound in the verifier's domain
Replay a signature between the owner's own smart and agent wallets	**Rejected** verifying contract is bound
Malleated signature (s replaced by `N − s`)	**Rejected**

Zeroed signature	Rejected
Signature over different contents, or another dApp's domain	Rejected
Tampered `authenticatorData` (user-verification flag dropped)	Rejected
Challenge swapped for another digest	Rejected
`webauthn.create` presented where `webauthn.get` is required	Rejected

On-chain verification — Arbitrum One (mainnet)

Test wallet `0x605A4C65dBa64bd26d9F7701e1e4Cda48c594A5c`, agent `0x38529C66F3cf22453D66B9E2A20FdF2676544aB4`.

DATE	OPERATION	RESULT
2026-07-16	Agent swap USDC→WETH through SwapRouter02, driven by the MCP server `0x4768603d38b03f9823d5d25e53fb03970b8e4d294ebee791d0ae9ef92e71a8ae`	OK USDC debited exactly, WETH to the wallet, fee 0.15% exact
2026-07-20	Agent swap through the Universal Router — an atomic 3-call batch crossing three different layers `0x6f831b37ae5db240256bf6bf99c8ce418edb75a1c38e37d6ed3e2c3a052a3bc0`	OK gas 419,124 · USDC −0.100000 · fee 0.000000078468831120 WETH = 0.15% exact · every recipient the wallet
2026-07-20	Same swap with `recipient` = attacker (simulated, no funds moved)	DENIED reverts before executing

The three calls of that batch each traverse a different guard: `USDC.approve(Permit2)` is case 2b and is checked against `allowedRecipients`; `Permit2.approve` is case 3 with `PIN_PROTOCOL` on the spender; `UniversalRouter.execute` is case 3 with `DEEP` and is decided by the validator.

Gate behaviour, measured before and after the validator's 48-hour activation:

REGISTRY.VALIDATE(...)	BEFORE ACTIVATION	AFTER ACTIVATION
Legitimate swap (recipient = wallet)	false	**true**
Theft attempt (recipient = attacker)	false	false
Command `0x40` (cross-chain bridge)	false	false
Target ≠ the bound router	false	false

The mask fix is therefore confirmed on mainnet, not only in tests, and the fail-closed property is visible in the "before" column: until the validator was active, even the legitimate swap was denied.

Gas

OPERATION	GAS
`createWallet`	`4,548,977`
`authorizeAgent`	`166,803`
`setCallPolicy`	`25,912`
`executeAsAgent` (case 3, policy checked)	`64,758`

C Static Analysis

Slither was run over the tree at each round. This appendix records the results and, as important, which results are false positives and why — so they are not re-investigated, and so the two virtual functions flagged as dead code are not deleted.

Static analysis — V5

Slither 0.11.3 was run over the V5 tree on 2026-10-26, before the review rounds. It found one real issue: an `allowance` read that copied however much data the token returned (a "return bomb", the same class as the uncapped `balanceOf` read flagged on V4). It was fixed; every read of a token's return data in V5 is now capped at 32 bytes and has a gas limit. The other results are by design (block timestamps for deadlines and expiry, one registry call per protocol item in a batch) or false positives. The V4 run's lessons were applied: the pragma is exact, and the two functions Slither marks as unused are kept.

Earlier generations

Slither 0.11.3 was run on both repositories prior to redeploy. It reported 39 results across 59 contracts; **none were actionable**. The table summarizes the categories and their disposition.

CATEGORY	EXAMPLES	DISPOSITION
By-design patterns	`block.timestamp` comparisons, inline assembly, low-level calls, calls-in-loop (batch)	Accepted
False positives	“uninitialized” mapping; reentrancy in `createWallet` (only an `emit` after trusted `setGuardians`)	No action
Dead code — `_feeNumerator()`	Flagged as unused	False positive — verified in use (see note)
Missing zero-check — factory `owner_`	Constructor sets owner without validation	Fixed
Cosmetic	Naming, literal digits, cyclomatic complexity	Accepted

> **Verification note — tool findings are not facts.** Slither marked `_feeNumerator()` as removable dead code. A source check showed it **is** used to compute the protocol fee (base 0.05% / agent override 0.15%); removing it would have broken fee logic. The finding was a false positive caused by virtual-function dispatch and was **not** acted upon.

Remediation applied

A defensive zero-address check — `if (owner_ == address(0)) revert ZeroOwner();` — was added to both factory constructors, synchronized across both repositories. Full suite green: **forge test = 128/128** in each repo. Slither was re-run on the V2 contracts after the gas hardening (Appendix D): **0 results**; the unit suite now stands at **131/131**.

Static analysis — Slither re-run on V3

Slither 0.11.3 was re-run over the V3 tree on 2026-07-27 (`slither . --exclude-dependencies`, 65 contracts, 100 detectors): **54 results — 3 High, 29 Low, 22 Informational**. Every High-impact result

was traced back to the source before being accepted or dismissed. One of them is real: the `reentrancy-eth` result is a confirmed HIGH finding, reproduced with a proof-of-concept test and still open at the time of writing (BVCC-01).

> **Read this run, not the V2 one.** The V2 pass reported "0 results", but only because the analyzer could not lower OpenZeppelin's transient-storage `ReentrancyGuard` to IR and bailed out (Appendix C). That zero was the tool giving up, not a clean bill of health. The V3 tree analyzes end to end — the detectors do reach `executeAsAgent`, as the reentrancy result below shows — so this is the static-analysis datapoint that carries weight, and its result classes line up with the 39 results of the V1 run.

DETECTOR	TARGET	VERDICT
`reentrancy-eth` confidence: medium	`executeAsAgent` writes `_currentAgent = 0` after `super.execute()`	**CONFIRMED — HIGH** — reproduced with a PoC test (BVCC-01)
`uninitialized-state` ×2 confidence: high	`_tokenTotalSpent`, `_tokenDailySpent`	**False positive** — both are `mappings`, which have no initializer and default to zero

D Previously Remediated — V1 & V2

These findings were reproduced and closed in generations that V4 and V5 supersede. They are retained with their evidence because wallets created by those factories are still live until their owners migrate, and because the remediations they describe are inherited by V4.

HIGH FIXED & REDEPLOYED

Description

An agent's per-token **daily** and **total** budgets could be bypassed entirely. The agent validation accounted spend for `transfer` but charged `approve` only against the per-transaction cap — not the daily or total budgets, and without restricting the spender. An agent could therefore `approve` an allowance (only per-tx-capped) and drain it with an external `transferFrom`, never touching the cumulative counters.

Proof of concept (executed on-chain, budgets exhausted at daily 2/2 · total 2/3)

STEP	ACTION	RESULT
1	`executeAsAgent → USDC.approve(B, 1)`	Passes — per-tx OK, daily ignored
2	As EOA: `USDC.transferFrom(wallet, B, 1)` (outside `executeAsAgent`)	1 USDC extracted; counters unchanged

A plain `transfer` of the same amount correctly reverted `TokenDailyLimitExceeded`; `approve` did not. Because `approve` had no daily limit, the `approve(1) → transferFrom(1)` cycle was repeatable — draining the wallet past both the daily (2) and total (3) caps. The PoC funds were returned to the wallet.

Remediation

In the `executeAsAgent` accumulation loop the condition was extended with `|| _isApprove(batch[i].callData)` so that `_applyTokenSpend` charges `approve` against the same daily/total caps and updates the counters. The choice is deliberately conservative (an `approve` overwrites the allowance, so it may over-count — the safe direction). Six regression tests were added; **5 fail without the fix, all pass with it.**

> **Bounded surface.** Tokens accept only `transfer` and `approve` by selector; `transferFrom` and `increaseAllowance` are blocked. ETH has no delegation primitive, so it is unaffected. `approve` was the sole vector — now closed and re-verified post-redeploy (see Appendix B, test #4 context).

Both factories changed bytecode (zero-check), producing new deterministic CREATE2 addresses on Arbitrum Sepolia. The frontend network configuration was updated accordingly.

CONTRACT	NEW ADDRESS (PATCHED)	PREVIOUS (DEPRECATED)
AgentWalletFactory	`0xc87aa10747A92B472EF6B36e190B84c897a2953e`	`0xe7ad…7e04`
SmartWalletFactory	`0xa5290A51a73903176e09C864E1542a07da67BD12`	`0x6890…F58d`

Privilege separation

- **Deployer (no control):** `0xA3e06B6443D911915bb4eD157832d43C25D6617D` — the factory's `owner` is set from a constructor argument, not `msg.sender`, so the deployer holds no privileges.
- **Admin / kill-switch owner:** `0x3145Bd5e2489d8bDdAb17F23F26F07Ac5aD55A4c` — immutable; the only privileged action is the one-way `kill()`.

Runtime sizes within the EIP-170 limit (AgentWallet 21,169 B; AgentWalletFactory 23,803 B — 773 B of headroom).

The base wallet supports social recovery: a 2-of-3 guardian quorum can rotate the owner passkey after a timelock. This was exercised end-to-end on wallet `0xEBc851bA9e8Ce32A2cb8eB99544F5F024f06E61d` and is now **complete**.

Process

- **Initiation (2026-06-06).** Recovery proposed toward a new owner passkey; guardians 1 and 2 (`0xa337…5908`, `0x8e06…2CcC`) approved — quorum 2/2.
- **Timelock.** On-chain `recoveryReadyAt` = 2026-06-08 17:24 UTC; throughout the window the owner retained the ability to cancel.
- **Execution (2026-06-08).** After the timelock elapsed, `executeRecovery()` was called from the wallet `/recover` UI. The on-chain `signer` rotated to the new passkey (X `0x303591…` / Y `0xb858ed…`), replacing the previous owner (X `0x070f7a…`).
- **Positive check.** The new owner immediately signed a real **19 USDC** transfer with the new Face ID — confirmed on-chain.

Negative test — the old passkey is neutralized

To confirm the rotation actually revoked the previous key, a transfer was attempted with the **old** passkey, whose WebAuthn credential is still present in the test authenticator. The signature is produced locally but must be validated on-chain against the current owner.

#	ACTION	EXPECTED	RESULT
1	Send 0.0001 ETH, UserOp signed by the OLD passkey	rejected at validation	FailedOp(0, AA24 signature error)
2	On-chain effect (ETH / USDC balances & nonce)	no change	Unchanged

> **Result.** Recovery completed successfully: ownership moved to the new passkey, the new key operates the wallet, and the old key is cryptographically rejected by the EntryPoint during signature validation — it can no longer move funds even though its local credential still exists.

HIGH (availability / overpayment) FIXED — V2 DEPLOYED 2026-06-11

Description

On Arbitrum mainnet, swaps through the wallet failed at gas estimation. The Case-3 fee logic scans calldata for token candidates and probes each one with `balanceOf`. Arbitrum system precompiles (`0x64–0x6f`) report bytecode (`0xfe`), so the code-length guard did not filter them — and calling them consumes **all forwarded gas** instead of reverting cheaply. The estimator kept raising the gas limit until ~32M so the probe would "fit"; that inflated limit is what the wallet would have prefunded (a 64× overpayment risk), and in practice the operation failed.

	BEFORE (BUG)	AFTER (V2 FIX)
Frontend / bundler gas estimate	~32,000,000	~400,000
Gas-burning candidate effect	burns all forwarded gas	capped at 100k, execution continues

Outcome	swap fails (or 64× overpayment)	normal swap, ~$0.02–0.05 on Arbitrum

Remediation (V2)

- **Gas-capped probes.** A shared constant `PROBE_GAS_CAP = 100_000` caps every `balanceOf` staticcall in both `_tryBalanceOf` (snapshot) and `_collectFeesOnIncrease` (fee collection). Worst case across a full scan is bounded to ~2.2M gas vs 32M before; real ERC-20s need well under the cap, so fee accounting is unaffected.
- **Fee collection can never break a user call.** If a token misbehaves after the swap, its fee is skipped instead of reverting the user's transaction.
- **Explicit accounting bound.** `executeAsAgent` now requires `totalEth ≤ type(uint128).max` before casting, removing a silent-truncation edge case for agents configured with no ETH limits.

Verification

- **forge test = 131/131** — regression tests added: a gas-burning candidate does not starve the swap; a reverting `balanceOf` is filtered non-fatally; a fat-return token is still snapshotted.
- **Slither 0.11.3 re-run on V2: 0 results.** (Tool note: Slither cannot generate IR for OpenZeppelin's transient-storage `ReentrancyGuard` internals — a known parser limitation, not a finding.)
- **EIP-170 sizes:** AgentWallet 21,214 B; AgentWalletFactory 23,848 B (728 B headroom); SmartWallet 13,432 B; SmartWalletFactory 16,030 B.

V2 deployment — deterministic CREATE2, same address on every chain

CONTRACT	ADDRESS (IDENTICAL ON ALL NETWORKS)
SmartWalletFactory V2	`0x230b7010529AB6977Dd8581B3eF018ef865BdEf1`
AgentWalletFactory V2	`0x8D9e24022777173AD6336e00884b6C87c7EF054c`

Deployed 2026-06-11 to Arbitrum One (42161), BNB Chain (56) and Arbitrum Sepolia (421614). Wallets created by pre-V2 factories do not receive the fix — new wallets must be created through the V2 factories.

Three issues in the wallet web application were fixed and synchronized to the public repository.

AREA	ISSUE	FIX
`lib/wallet.ts`	Credential recovery queried only the base factory event, so re-opening an *agent* wallet by address failed (“no active wallet”).	Also query `agentFactory` / `AgentWalletCreated`.
Agents page	Per-token limits shown as text only.	Read `getTokenSpent` on-chain; render Daily/Total progress bars (verified live).
Deploy flow	App-supplied gas under-estimated the ~21 KB agent-wallet deploy on Arbitrum, causing signing failures.	Defer to the wallet’s network estimate.

E Remediation Timeline

ROUND	DATES	FOCUS	OUTCOME
V1	2026-06-06 → 06-08	Agent permission model, first static analysis	BVCC-A1 found, fixed, redeployed; permission battery and guardian recovery verified end-to-end
V2	2026-06-11 → 06-12	Mainnet gas behaviour	BVCC-A2 found and fixed; deterministic multichain deployment across six networks
V3	2026-07-21	Call-policy layer and validator registry	Fund-exfiltration path closed; BVCC-02 and BVCC-12 caught inside the work, before deployment
V4	2026-07-27 → 07-28	Post-release review, second tooling pass	BVCC-01, BVCC-04 to BVCC-08 and BVCC-11 found and fixed; deployed and verified on six networks
—	2026-07-28 → 07-29	Validator activation	Deep-validation layer completed on all six networks; v4 liquidity cycle exercised on mainnet
V5	2026-10-23 → 10-26	Design and code	Specification with seven guarantees; core written; 146 tests including six-network deployment forks; Slither return bomb fixed
V5 round 1	2026-10-27	Full review with an AI tool	BVCC-14 to BVCC-20 found; validator required on every protocol call; charged inflows
V5 rounds 2–3	2026-10-28	Fix verification, Uniswap paths on forks	BVCC-21 to BVCC-23 found and fixed in the validators
V5 round 4 · red team	2026-10-29	Hooks; attacks from inside and outside	BVCC-24 to BVCC-30: five fixed, BVCC-25, BVCC-29 and BVCC-30 accepted
V5 round 5	2026-10-29	Red-team fixes, freeze readiness	No new findings; code frozen (176 tests at that point)
—	2026-10-30 → 11-02	Deployment	Arbitrum Sepolia, then the five mainnets; Uniswap v3 and wrapped-native validators added and reviewed (no findings)
—	2026-11-01 → 11-08	Validator activation and testing	30 of 30 validators active after the 48-hour delays; full agent battery and a V5 recovery on Arbitrum Sepolia
—	2026-11-11	Verification and migration	72 of 72 contracts source-verified; a V4 wallet migrated to V5 on BNB Chain; this edition

The V3 deployment record, build freeze, off-chain layer and migration notes follow.

V3 deployment — deterministic CREATE2

CONTRACT	ADDRESS (IDENTICAL ON ALL SIX NETWORKS)
SmartWalletFactory V3	`0xD42F61AA856A4f47885Ecd2D0ce119411d53C192`
AgentWalletFactory V3	`0xd866a7563cDaC9F71423be3332b62c329C676064`

ValidatorRegistry	`0x5e371D54AC97a57B0a99145Ed04A3c9fA07850C2`
HookRegistry	`0x551C6e7ABdA04a110790888e711198f25621b066`

Networks: Arbitrum One (42161), Ethereum (1), BNB Chain (56), Base (8453), Polygon (137) and Arbitrum Sepolia (421614). All four contracts were re-checked on 2026-07-27 through the explorers' API: present, byte-identical and source-verified on all six chains.

Universal Router validators are bound to one router each, so their addresses differ per network:

NETWORK	UR VALIDATOR	REGISTRY STATUS
Arbitrum One	`0xD1522594e185C882bfDEc6FFD4DE8a53F69AF0Ca`	Active
Ethereum	`0x3615a0Fc0663C0201B543deC9816d5Ef30a82ffb`	Active
BNB Chain	`0x6945f861e0D7FCD566739000f3fDC1D44896D815`	Active
Base	`0x4A2Fc73AAdEfC84513defaa0c444d2150AC6A6D8`	Active
Polygon	`0xC1C584E4149eD2EFe20DA0155b1B8257232102fF`	Active
Arbitrum Sepolia	`0x87550034627966e32d82E3b8AdB3f19c62E8d86E`	Active

Status read directly from `validators(router)` on each chain, last on 2026-07-28 after Arbitrum Sepolia was activated. A deployed-but-unactivated validator is inert in the safe direction: the corresponding DEEP calls fail closed. Per-chain provenance manifests (addresses plus router and validator codehashes) live in `contracts/deployments/`. The Uniswap v4 `PositionManager` validators were activated on all six networks in the same round — see Appendix A for the addresses and the per-network binding check, and Appendix D for the liquidity cycle run against them.

Build freeze

Deterministic CREATE2 addresses require a frozen build: solc **0.8.36**, `via_ir`, `optimizer_runs = 50`, EVM `cancun`. That compiler was chosen deliberately to stay clear of the 0.8.28–0.8.33 via-IR pipeline bug. The agent factory's runtime size is **24,283 bytes**, leaving 293 bytes of headroom under the EIP-170 limit — any future addition must be measured against it.

Off-chain layer — SDK and MCP 0.2.0

The client libraries were made policy-aware so that the on-chain guard is not discovered only at broadcast time. `getCapabilities` now gates a swap on the registered policies *and* the validator registry rather than on the permission struct alone, and reports the wallet version plus any missing policies; the four V3 errors decode into actionable messages, distinguishing "no validator active" (do not retry) from "the validator rejected this calldata" (fixable). `setCallPolicy` is deliberately **not** exposed by the SDK: policies are written by the owner's passkey, never by an agent's key. Aave lending and its planners ship through the same catalog, with the beneficiary always pinned to the wallet.

Migration

V3 wallet bytecode differs from V2, so the CREATE2 addresses differ: existing users create a new wallet and move their funds, exactly as in the V1→V2 migration, after which the V2 factories are killed. Future capabilities — registering further validators, adding protocol presets — are bytecode-neutral and will not force another migration.

F Key Identifiers & Addresses

ITEM	VALUE
Network	Arbitrum Sepolia (421614)
Patched wallet	`0x1ED261BA084c4E6117daf289f3ca233BBA3cF0C3`
Agent (role B)	`0x38529C66F3cf22453D66B9E2A20FdF2676544aB4`
AgentWalletFactory (Sepolia, pre-V2)	`0xc87aa10747A92B472EF6B36e190B84c897a2953e`
SmartWalletFactory (Sepolia, pre-V2)	`0xa5290A51a73903176e09C864E1542a07da67BD12`
SmartWalletFactory V2 (all networks)	`0x230b7010529AB6977Dd8581B3eF018ef865BdEf1`
AgentWalletFactory V2 (all networks)	`0x8D9e24022777173AD6336e00884b6C87c7EF054c`
SmartWalletFactory V3 (all networks)	`0xD42F61AA856A4f47885Ecd2D0ce119411d53C192`
AgentWalletFactory V3 (all networks)	`0xd866a7563cDaC9F71423be3332b62c329C676064`
SmartWalletFactory V4 (all networks)	`0xfd105197109244483b5f870501326E6faec9F93c`
AgentWalletFactory V4 (all networks)	`0xf3A61F9d64d45362E149A111289546523BCd26a6`
SmartWalletFactory V5 (all networks)	`0x2672951DC1E3f865BA6Fc98e5509dc4B23B260a2`
AgentWalletFactory V5 (all networks)	`0xE36fb66c21C38536454C1a58fBC5EB1Cb455aD52`
SmartWallet logic V5 (all networks)	`0xc9dE89d54beB786bc916131CA6c8c5374507Bd15`
AgentWallet logic V5 (all networks)	`0x360f991B23a3D5202Dc422292697aB07dA279cFA`
Agent logic library V5 (all networks)	`0x7504bf3c1230eF99eaEFAc03287c8F036CBb6D0E`
ValidatorRegistry V5 (all networks)	`0x7269334AE74DCE4e607eec36268383DD35f28350`
HookRegistry V5 (all networks)	`0xEaEaC7F6071f7F576Be2aD97898Dd0c0F51BE78A`
V5 validators	Per network, listed in §5
Fee wallet V5 (all networks, constant)	`0x3e3eb089169a7315a994947465ce5f5FC3A307D4`

V5 migration test (BNB Chain)	`0x566669C6cC8FDD33c08Ed56A51c23ec1972A4936`
ValidatorRegistry (all networks, V3 & V4)	`0x5e371D54AC97a57B0a99145Ed04A3c9fA07850C2`
HookRegistry (all networks, V3 & V4)	`0x551C6e7ABdA04a110790888e711198f25621b066`
v4 LP test position (Arbitrum One)	`tokenId 190877 — minted and burned by an agent`
V3 test wallet (Arbitrum One)	`0x605A4C65dBa64bd26d9F7701e1e4Cda48c594A5c`
Governance admin (hardware wallet)	`0x3145Bd5e2489d8bDdAb17F23F26F07Ac5aD55A4c`
V2/V3/V4/V5 networks	Arbitrum One · Base · BNB Chain · Ethereum · Polygon · Arbitrum Sepolia
V3/V4/V5 build	solc 0.8.36 · via-IR · optimizer 50 · EVM cancun (frozen for CREATE2); V5 without metadata hash
Tooling	Slither 0.11.3 · Foundry (forge 1.5.1) · viem 2.x

Confidential — Block Venture Chain Capital. Prepared for internal review and repository inclusion. The V1 permission battery was executed on a public testnet (Arbitrum Sepolia); the V3 call-policy layer and the v4 liquidity cycle were additionally verified on Arbitrum One mainnet. V2, V3 and V4 factories are deployed on Arbitrum One, Ethereum, BNB Chain, Base, Polygon and Arbitrum Sepolia, with the Universal Router and v4 PositionManager validators registered on all six. V5 is deployed on the same six networks with its own registries and five validators per network, all active, and every V5 contract is source-verified.

G Process Notes

Recorded because a review that only lists what it found overstates its own reliability.

Three lessons are worth recording alongside the findings. The reentrancy was dismissed once, on plausible reasoning about EOAs, before a proof-of-concept contradicted it: assumptions about the shape of an address deserve a test, not an argument. The most serious issues here were each found by looking again at code that had already passed a review — guardian squatting came from a pass with different tooling, credential rotation from a follow-up of that same pass, and two more from attacking recovery directly. And the static-analysis count moved by one across all of it: it did not rise while the reentrancy was exploitable for three generations, and it did not fall when the fix closed it. Tools find patterns; the findings here came from reading code and asking what an attacker would do with it. The contracts remain internally reviewed and are not externally audited.

V5 added three more. **Fixes breed neighbours.** Rounds 2, 3 and 4 and the red team each found a new path one step from the previous fix, all in the Uniswap position validators: an owner check on adding but not removing, a check on the target position but not on its approvals, a hook check on swaps but not on withdrawals. The rules only held once they were made symmetric across every action. A fix to a validator should be followed by the question "which sibling action did this not touch?". **Balances cannot see everything.** Measuring outflows closed BVCC-03, but value can also move as liquidity between NFTs, as collateral, or through a hook, with no token leaving the wallet. Each of those needed its own rule, and the ones that cannot be ruled out are written down as trust assumptions. **Write the specification first.** The V5 rounds were checked against seven written guarantees, which turned arguments about intent into tests that pass or fail. Where the specification was ambiguous (G-4 for protocol payouts, BVCC-16), the finding forced a decision instead of a debate. Round 5 also noted that its auditor had no copy of the round-4 code to compare against; the team made that comparison, and the frozen code is now tied to a commit, its hashes, its tests and its addresses.